

Hochschule RheinMain
Fachbereich Design Informatik Medien
Studiengang Angewandte Informatik

Bachelor-Arbeit
zur Erlangung des akademischen Grades
Bachelor of Science - B.Sc.

**Evaluierung der Fehlerkorrektur und
authentifizierten Verschlüsselung des MAGIC-Modus
auf einem Mikrocontroller mit externem Speicher**

vorgelegt von

Jonas GILBERT
Matrikelnummer 1288740
Auf dem Langwehr / 2
65326 Aarbergen

am

02.05.2025

Referent:

Prof. Dr. Marc STÖTTINGER

Korreferent:

Julian WÄLDE M. Sc.

Formales

Eigenständigkeitserklärung (gem. ABPO, Ziff. 4.1.5.4 bzw. RPO, §19.6)

Ich bin verantwortlich für die Qualität des Inhalts dieser Arbeit, den ich mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken, technischen Inhalte und ähnliches in meinen Ausführungen habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen. Alle weiteren Inhalte der Arbeit (Textteile, Abbildungen, Tabellen etc.) ohne entsprechende Verweise stammen im urheberrechtlichen Sinn von mir. Ich versichere außerdem, dass ich die Bachelor-Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich versichere außerdem, dass ich zur Erstellung des Quellcodes für diese Arbeit keine KI-generierten Inhalte verwendet habe. Für die Texterstellung der vorliegenden Arbeit habe ich eine KI-basierte Rechtschreibprüfung genutzt, die mich bei der Rechtschreibung und Grammatik während des Schreibens unterstützt hat. Bei der Erstellung des Abstracts in Englisch haben mich Übersetzungsprogramme unterstützt.

Die vorliegende Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer anderen Prüfungsbehörde vorgelegt. Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte als Täuschungsversuch gelten und prüfungsrechtliche Konsequenzen haben kann. Insbesondere kann es dazu führen, dass die Leistung nicht bestanden ist und dass bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation droht.

Ort, Datum

Unterschrift Studierender

Aarbergen, 02.05.2025



Hiermit erkläre ich mein Einverständnis mit den im Folgenden aufgeführten Verbreitungsformen dieser Bachelor-Arbeit:

Verbreitungsform	ja	nein
Veröffentlichung des Titels der Arbeit im Internet	X	
Veröffentlichung der Arbeit im Internet	X	

Ort, Datum

Unterschrift Studierender

Aarbergen, 02.05.2025

Jonas Gilbert

Abstract

Deutsch

In dieser Bachelorarbeit soll der MAGIC-Modus auf einem Mikrocontroller evaluiert werden. Dieser Modus stellt gleichzeitig eine authentifizierte Verschlüsselung und Fehlerkorrektur zur Verfügung. MAGIC wurde implementiert und mit dem Hamming-Code bezüglich der Fehlerkorrektur verglichen. Für die Evaluierung wurden die Taktzyklen und der Speicherverbrauch auf dem Mikrocontroller betrachtet. Die Evaluierungsergebnisse zeigen, dass der Hamming-Code in Bezug auf die Fehlerkorrektur deutlich schneller ist. Vor allem da ein beliebiger Hashwert im Vergleich zu MAGIC verwendet werden kann. Es werden jedoch mindestens 128 Bits zusätzlicher Speicher in Vergleich zum MAGIC-Modus benötigt. Jedoch kann MAGIC in der Theorie mehr Fehler erkennen und korrigieren als der Hamming-Code. Dies zeigt, dass man nach aktuellem Stand weiterhin den Hamming-Code verwenden sollte. Jedoch zeigt die Arbeit auch, dass durch MAGIC durchaus die Komplexität verringert wird. Sollte man schneller einen Hashwert beim MAGIC-Modus finden, kann es in der Praxis problemlos eingesetzt werden.

English

In this bachelor thesis, the MAGIC mode should be evaluated on a microcontroller. This mode supports authenticated encryption and error correction simultaneously. MAGIC was implemented and compared with the Hamming code regarding the error correction. For the evaluation on the microcontroller, the clock cycles and the memory usage were viewed. The evaluation results prove that the Hamming code is much faster in terms of error correction because an arbitrary hashkey can be used to generate a MAC. For this, MAGIC cannot use an arbitrary hashkey. But at least 128 bits more are required compared to the MAGIC mode. In theory, MAGIC can detect and correct more errors than Hamming code. The results indicate that the Hamming code should still be used according to the current status. This thesis shows as well that the complexity with MAGIC is reduced. When it is possible to find a hashkey more efficient in the MAGIC mode, you can use it without problems in the real world.

Inhaltsverzeichnis

1	Einführung	1
1.1	Problemstellung und Motivation	1
1.2	Ziele der Arbeit	2
1.3	Aufbau der Arbeit	3
2	Grundlagen	4
2.1	Arithmetik in endlichen Erweiterungskörpern	4
2.2	Der MAGIC-Modus	5
2.3	Salsa20	7
2.4	Hamming-Code	9
3	Konzept und Vorgehensweise	11
4	Implementierung	15
4.1	Programmaufbau	15
4.2	Implementierung von 128-Bit großen Zahlen	16
4.3	Arithmetik in endlichen Erweiterungskörpern	19
4.4	Umsetzung der Ver- und Entschlüsselung durch Salsa20	21
4.5	Realisierung des MAGIC-Modus	23
4.6	Hamming-Code pro Block und über alle Blöcke	28
4.7	Arbeiten mit dem Mikrocontroller	32
4.8	Die Funktionsweise der Tests	33
5	Evaluierung	37
5.1	Die Bestimmung der Takte	37
5.2	Vorgehensweise zur Bestimmung der Takte für eine Operation	38
5.3	Performance des MAGIC-Modus	39
5.4	Performance des Hamming-Codes	43
5.5	Performance der Ver- und Entschlüsselung	48
5.6	Speicherkosten und Mehraufwand der beiden Verfahren	49
6	Schlussfolgerung	51
6.1	Die Performance im Vergleich	51
6.2	Speicherkosten und Mehraufwand im Vergleich	52

7 Fazit und Ausblick	53
7.1 Fazit	53
7.2 Ausblick und mögliche Weiterentwicklungen	54
Informationen zum Quellcode und zu Quellenangaben	II
Abkürzungsverzeichnis	III
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Quellcodeverzeichnis	VI
Literatur	IX

Kapitel 1

Einführung

Die folgende Bachelorarbeit befasst sich in großen Teilen mit dem Message Authentication, Galois Integrity and Correction (MAGIC)-Modus, der von Kounavis u. a. entwickelt wurde. Mithilfe von MAGIC lassen sich Nachrichten gleichzeitig authentifiziert verschlüsseln und korrigieren [Kou+20].

Zunächst wird dieser Modus auf einem beliebigen Computer implementiert und anschließend auf einen Mikrocontroller portiert. Auf diesem Mikrocontroller findet die Evaluierung statt.

1.1 Problemstellung und Motivation

Heutzutage werden die authentifizierte Verschlüsselung und die Fehlerkorrektur von Code in Computersystemen oft getrennt implementiert. Dadurch entsteht ein unnötiger Mehraufwand und der Computer braucht mehr Ressourcen. Des Weiteren nimmt auch die Komplexität dadurch zu. Man kann die authentifizierte Verschlüsselung und Fehlerkorrektur auch nicht weglassen, da sonst große Schwachstellen entstehen könnten. Die authentifizierte Verschlüsselung stellt sicher, dass potenzielle Angreifer die Daten nicht sehen können. Man braucht den richtigen Schlüssel, um an die Daten zu kommen. Ebenfalls kann ein Empfänger der Daten damit sicherstellen, ob diese vom erwartbaren Sender stammen. Nun könnte ein Angreifer die verschlüsselten Daten immer noch verändern. Die manipulierten Daten werden aber ohne weitere Verfahren nicht erkannt. Diese könnten auch durch den Random Access Memory (RAM) fehlerhaft werden. Zunächst müssen die Fehler in den Daten erkannt werden. Anschließend kann man diese Fehler eventuell korrigieren, um die richtigen Daten wiederherzustellen. Man könnte für die authentifizierte Verschlüsselung und Fehlererkennung/Fehlerkorrektur zwei getrennte Verfahren verwenden. Anstatt dessen ist die Lösung, diese zwei Verfahren zu einem Verfahren zu kombinieren. Dadurch spart man Ressourcen und die Komplexität nimmt ab. Dieses kombinierte Verfahren nennt sich MAGIC [Kou+20] [Lam+24].

MAGIC soll in der Bachelorarbeit auf einem Mikrocontroller mit externen Static Random Access Memory (SRAM) implementiert und ausgeführt werden können. Zusätzlich soll es dort auch evaluiert werden. Zum Zeitpunkt der Erstellung der Bachelorarbeit existiert noch keine Implementierung von MAGIC, weshalb dies mit dieser Arbeit realisiert wurde. Dabei wird es auf einem Mikrocontroller ausgeführt, da diese sich immer öfter im Alltag wiederfinden. Unter anderem in Internet of Things (IoT)-Geräten, Industrieanlagen und Autos. Dort kann man physikalische Angriffe auf dem externen Speicher nicht mehr ausschließen. Man muss solche Geräte sichern, was mit einer erhöhten Komplexität und höherem Speicheraufwand einhergeht. Wie im vorherigen Abschnitt dargestellt, verringert der MAGIC-Modus vermutlich die Komplexität und den Speicheraufwand im Vergleich zu herkömmlichen Verfahren. Demzufolge wird dies in der Bachelorarbeit auf einem Mikrocontroller mit externem SRAM getestet.

1.2 Ziele der Arbeit

Ziele der Bachelorarbeit beinhalten, dass man den MAGIC-Modus zu Beginn in der Programmiersprache C auf einem beliebigen Computer implementiert. Anschließend soll die C-Implementierung auf ein Ballistic Gel Devboard (Board) portiert werden. Dort soll die Implementierung dann funktionsfähig laufen. Das Board besitzt einen Cortex-M3 Prozessor mit einem externen SRAM.

Ebenso sollen die Verbrauchskosten dieser Implementierung für eingebettete Systeme anhand von Bewertungskriterien wie Taktzyklen und Speicherverbrauch bewertet werden. Hier soll herausgefunden werden, wie viele Takte/Taktzyklen pro Operation auf dem Mikrocontroller benötigt werden. Zu den Operationen gehören die Verschlüsselung, Entschlüsselung, Tokengenerierung (Taggenerierung) und das Finden eines passenden Hashwertes. Dabei soll für die Evaluation auch die Anzahl der Blöcke und die Anzahl der korrigierbaren Bits (Schwellenwert) variiert werden.

Zum anderen sollen die Speicherkosten und der Mehraufwand, der durch den MAGIC-Modus entsteht, untersucht werden. Dazu gehört, wie viel Speicherplatz mehr durch MAGIC benötigt wird und wie das Verhältnis zwischen Overhead (Token/Tag) und gespeicherten Daten ist. Außerdem soll geprüft werden, inwiefern es Auswirkungen auf die Fehlererkennung und Sicherheit hat, wenn weniger Blöcke mit einem Token/Tag abgesichert sind. Insbesondere wird noch betrachtet, wie sich eine reduzierte Anzahl von Blöcken auf die Fehlerkorrektur und das Finden von Hashwerten auswirkt.

Zu guter Letzt soll der Hamming-Code evaluiert und mit den Ergebnissen aus der Evaluationen des MAGIC-Modus verglichen werden. Es soll anschließend eine Aussage in Bezug auf die Fehlerkorrektur gemacht werden. Man kann dann aussagen, wie gut der MAGIC-Modus auf einem solchen Board funktioniert und ob MAGIC lohnenswert ist.

1.3 Aufbau der Arbeit

Zu Beginn befindet sich in Kapitel 2 das Hintergrundwissen, welches man zum Verstehen der Arbeit benötigt. Das Wissen orientiert sich an Bachelorabsolventen eines Informatikstudiengangs. Vor allem wird sich mit dem MAGIC-Modus und dem Hamming-Code befasst. Danach folgt in Kapitel 3 die Vorgehensweise und ein Konzept der Implementierung. Es wird dort gezeigt, was die Ziele und Vorhaben hinter bestimmten Dingen sind. Außerdem wird dargestellt, wie man mit dem Mikrocontroller arbeiten kann. In Kapitel 4 wird vor allem auf die Implementierung des MAGIC-Modus eingegangen. Dazu gehört unter anderem auch die Arithmetik in endlichen Körpern. Des Weiteren werden in diesem Kapitel die erstellten Tests erläutert. Im Anschluss werden in Kapitel 5 die Evaluierungsergebnisse in Form von Tabellen vorgestellt und erläutert. Außerdem wird gezeigt, wie die Takte auf dem Mikrocontroller ermittelt wurden. In Kapitel 6 werden die Ergebnisse aus dem vorherigen Kapitel gedeutet und miteinander verglichen. Im letzten Kapitel 7 wird ein Fazit gezogen und Entwicklungspotenziale vorgestellt. Auf den letzten Seiten befindet sich eine Danksagung und die jeweiligen Verzeichnisse. Des Weiteren wird dort erläutert, wie die Quellenangaben zu verstehen sind.

Kapitel 2

Grundlagen

Zu Beginn des Kapitels wird auf die Arithmetik in endlichen Erweiterungskörpern eingegangen. Diese nutzt MAGIC, um mit großen Zahlen zu rechnen. Anschließend wird der MAGIC-Modus erklärt und danach auf Salsa20 eingegangen. Salsa20 wird als Ver- und Entschlüsselungsverfahren in dieser Arbeit eingesetzt. Zu guter Letzt gibt es noch eine kurze Einführung in Hamming-Codes.

2.1 Arithmetik in endlichen Erweiterungskörpern

Ein endlicher Körper ist im Allgemeinen eine endliche Menge von Elementen, in der man addieren, subtrahieren, multiplizieren und invertieren kann. Die Anzahl der Elemente nennt man Ordnung. Ein endlicher Körper existiert nur, wenn die Ordnung q eine Primzahlpotenz der Form p^m ist. Dabei ist p eine Primzahl und m eine natürliche Zahl. Alle endlichen Körper mit $m > 1$ nennt man endliche Erweiterungskörper und werden in der Form $GF(p^m)$ notiert [PP16].

In den Erweiterungskörpern kann man die Elemente nicht als natürliche Zahlen darstellen. Für die Repräsentation der Elemente werden Polynome genutzt und die Polynomarithmetik zum Rechnen verwendet. Wenn der Erweiterungskörper die Form $GF(2^m)$ besitzt, werden die Elemente als Polynome in $GF(2)$ aufgefasst. Die Polynome haben hierbei den maximalen Grad von $m - 1$. Man kann die Polynome auch als Bitstrings der Länge m sehen. Wenn das Polynom $x^4 + 1$ lautet, dann ist der entsprechende Bitstring 10001_2 [PP16].

Um Elemente zu **addieren** oder **subtrahieren**, werden jeweils die zwei gleichen Koeffizienten der Polynome miteinander addiert. Anschließend muss von der Addition mod 2 gerechnet, da die Polynome in $GF(2)$ bleiben müssen. Dies kann man auch als eine XOR-Operation zwischen den zwei Bitstrings auffassen. Um Elemente zu **multiplizieren**, findet eine Polynommultiplikation zwischen den zwei Polynomen statt. Sollte nun das entstandene Polynom außerhalb des Erweiterungskörpers liegen, hat es einen

höheren Grad als $m - 1$. Es muss durch ein irreduzibles Polynom geteilt werden, um es wieder in den Erweiterungskörper zu bringen. Ein irreduzibles Polynom kann man wie eine Primzahl betrachten. Um das **inverse Element** bezüglich der Multiplikation zu finden, gibt es für kleine Erweiterungskörper häufig schon vorberechnete Tabellen. Jedoch kann das inversen Elemente auch direkt berechnet werden. Hierzu wird zum Beispiel der erweiterte euklidische Algorithmus verwendet [PP16].

Der MAGIC Modus nutzt den endlichen Erweiterungskörper der Form $GF(2^{128})$ mit dem irreduziblen Polynom $x^{128} + x^7 + x^2 + x + 1$ [Kou+20]. In diesem Körper finden alle Operationen statt. Außerdem wird in dieser Arbeit der kleine Satz von Fermat mit einer speziellen Potenzierung verwendet, um inverse Elemente zu finden. Mehr Details hierzu in Kapitel 3.

2.2 Der MAGIC-Modus

Der MAGIC-Modus wurde im Jahr 2021 von Michael Kounavis u. a. veröffentlicht. MAGIC basiert vor allem auf dem Galois Counter Mode (GCM) und Galois Message Authentication Code (GMAC) Operationsmodus für Blockchiffren. Der MAGIC-Modus hat den Vorteil, dass gleichzeitig authentifizierte Verschlüsselung und Fehlerkorrektur stattfinden können. Dazu wird der erzeugte Message Authentication Code (MAC) für die authentifizierte Verschlüsselung auch als Paritätswert für die Fehlerkorrektur verwendet. Der MAC wird in der weiteren Arbeit als Token bezeichnet [Kou+20].

Zunächst gibt es eine Nachricht, die in n (Anzahl der Blöcke) N (Anzahl der Bits pro Block) Bit große Blöcke aufgeteilt wird. Diese werden als $M_1, M_2, \dots, M_n$ bezeichnet und werden mit einem beliebigen Verschlüsselungsverfahren verschlüsselt. Dabei entstehen n verschlüsselte N Bit-Blöcke und werden als $C_1, C_2, \dots, C_n$ bezeichnet. Daneben gibt es beim MAGIC-Modus einen Hashwert H und authentifizierte Daten D , die jeweils N -Bit groß sind. Der Hashwert, die authentifzierten Daten und die verschlüsselten Blöcke sind dazu da, um einen Token T zu erzeugen. Der Token T kann für die Authentifizierung und die Fehlerkorrektur verwendet werden. Dazu werden all diese Werte folgendermaßen mit der Gleichung 2.1 in einem endlichen Erweiterungskörper der Form $GF(2^N)$ addiert und multipliziert [Kou+20].

$$T = E_K(D + C_1 \cdot H^1 + C_2 \cdot H^2 + \dots + C_n \cdot H^n) \quad (2.1)$$

Dazu wird das gleiche Verschlüsselungsverfahren verwendet, das die verschlüsselten Blöcke erzeugt. Jedoch muss ein anderer Schlüssel K und eine andere Nonce verwendet werden. Durch die nochmalige Verschlüsselung in Gleichung 2.1 wird sichergestellt, dass der Hashwert nicht von einem Angreifer trivial berechnet werden kann. Diesen berechneten Token kann man als eine Art MAC und gleichzeitigen Paritätswert für die Fehlerkorrektur sehen. Die dargestellten Berechnungen kann man sich in der folgenden

Abbildung 2.1 visuell verdeutlichen [Kou+20].

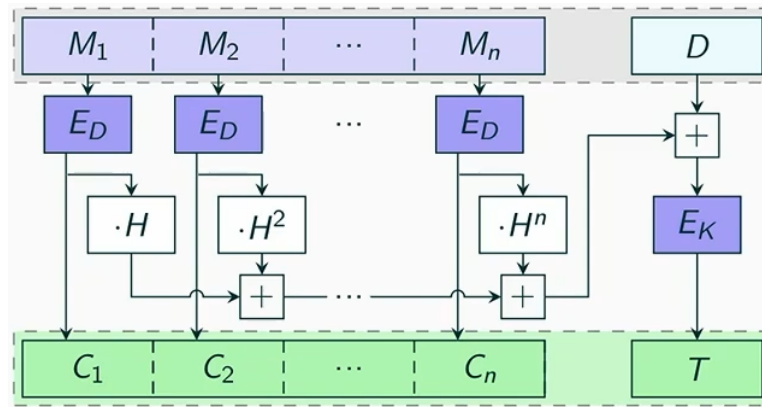


Abbildung 2.1: Die Tokengenerierung des MAGIC Modus [USE24].

Um Fehler mithilfe von T richtig korrigieren zu können, muss der Hashwert H bestimmte Bedingungen erfüllen. Zu Beginn wird ein Schwellenwert T_{th} definiert, der angibt, wie viele Bitfehler maximal in einem beliebigen Block C_j korrigiert werden können. Daneben gibt es einen Fehlervektor¹ e , der N -Bit groß ist. Zu guter Letzt wird noch eine Funktion $HW()$ genutzt, um das Hamming-Gewicht von einem beliebigen Wert zu bestimmen. Der Hashwert H wird zufällig ausgewählt [Lam+24] und folgendermaßen mit der Gleichung 2.2 überprüft [Kou+20].

$$(\forall e : e \neq 0, HW(e) \leq T_{th}) \wedge (\forall i \in [1, n]) : \quad (2.2)$$

$$HW(H^i \cdot e) > T_{th}, HW(H^{-i} \cdot e) > T_{th}$$

Bei der Gleichung 2.2 wurde sich am Voodoo-Paper [Lam+24] orientiert. Dort geht i bis einschließlich n und nicht wie im MAGIC-Paper [Kou+20] bis $n - 1$. Es wird die Bedingung $HW(S) > T_{th}$ dadurch sichergestellt und somit hat jedes Syndrom S eines korrigierbaren Fehlers immer einen Wert über T_{th} [Lam+24]. Ein Syndrom ist dabei ein Wert, um Fehler anzuzeigen.

Für eine mögliche Korrektur von Bitfehlern in einem Block oder dem Token T wird ein Syndrom S gebildet. Um Fehler zu korrigieren, darf die Anzahl der Bitfehler in einem Block oder im Token nicht größer als der Schwellenwert T_{th} sein. Das Syndrom S wird berechnet, indem T entschlüsselt und mit dem Hashwert und verschlüsselten Blöcken durch Gleichung 2.3 addiert wird [Kou+20].

$$S = D + C_1 \cdot H^1 + \dots + C_n \cdot H^n + E_K^{-1}(T) \quad (2.3)$$

Sollte nun ein Block fehlerhaft sein, wird das Syndrom S zu $S = e_j \cdot H^j$. Dabei steht j für den Index des fehlerhaften Blocks. Anschließend wird basierend auf S jeweils Syn-

¹In jeder Stelle, an der ein Bitfehler vorhanden ist, ist das Bit gesetzt. An den entsprechenden anderen Stellen nicht. Insgesamt gibt es N viele Bits.

dromwerte S_i pro Block gebildet, indem $S_i = S \cdot H^{-i}$ berechnet wird. Sollte S_i einen fehlerhaften Block anzeigen, gilt $S_i = e$ und dann ist $S_i = S_j$. Ansonsten gilt $S_i = e \cdot H^{j-i}$, wenn S_i einen korrekten Block anzeigt. Anhand von Gleichung 2.2, kann man nun sehen, dass der fehlerhafte Block C_j erkannt wird. Um diesen nun zu korrigieren, muss man C_j mit S_j im endlichen Körper addieren, also eine XOR-Operation ausführen, um C_j zu korrigieren. In der Gleichung 2.4 sieht man diese Operation, um einen fehlerfreien Block zu haben [Kou+20].

$$C_{correct} = C_j + S_j \quad (2.4)$$

Um einen fehlerhaften Token T zu korrigieren, wird dieser durch die Gleichung 2.1 neu berechnet. Sollten dabei alle verschlüsselten Blöcke C korrekt sein, ist dieser Token auch korrekt. Sollte aber mehrere Blöcke fehlerhaft sein, ist der Token nicht korrekt. MAGIC kann eben nur Fehler in maximal einem Block korrigieren. Ob ein Token korrigiert werden kann, wird durch die Gleichung 2.5 realisiert [Kou+20].

$$HW(T_{old} + T_{new}) \leq T_{th} \quad (2.5)$$

Wenn die verschlüsselten Blöcke korrekt sind, ergibt die Addition den Fehlervektor. Dieser muss laut Gleichung 2.2 kleiner oder gleich dem Schwellenwert T_{th} sein. Ist die Überprüfung in Gleichung 2.5 nicht erfolgreich, kann kein Token T oder Block C korrigiert werden [Kou+20].

2.3 Salsa20

Der MAGIC-Modus braucht ein Ver- und Entschlüsselungsverfahren. Zu diesem Zweck wurde in dieser Arbeit Salsa20 verwendet und implementiert. Die Gründe dazu kann man in Kapitel 3 nachlesen.

Salsa20 ist eine Stromchiffre und erzeugt einen 2^{70} großen Bytestrom, indem ein 256-Bit großer Schlüssel, eine 64-Bit große Nonce und eine Blocknummer genutzt werden. Jeder erzeugte Block im Strom ist 512-Bit groß. Um eine Nachricht zu verschlüsseln, findet eine XOR-Operation zwischen der zu verschlüsselnden Nachricht und dem 512-Bit großen Block statt. Dazu werden die ersten 512-Bits der Nachricht genutzt. Reicht der Block für die Nachricht nicht aus, wird ein weiterer Block erzeugt und mit dem nächsten 512-Bit der Nachricht verschlüsselt. Dies geht immer so weiter, bis alle Nachrichtenblöcke aufgebraucht sind. Der restliche Strom an erzeugten Blöcken wird nicht weiter beachtet. Die Entschlüsselung funktioniert mit gleichem Schlüssel und gleicher Nonce genau so. Es wird anstatt des Klartextes die verschlüsselte Nachricht genutzt [Ber08].

Ein 512-Bit-Block kann als ein Array mit 16 32-Bit großen Feldern angesehen werden. Um einen Block zu erzeugen, werden alle Felder an bestimmten Positionen mit

dem Schlüssel, der Nonce, der Position im Strom und konstanten Werten² im Little-Endian Format initialisiert [Ber08]. Wenn der Block für die XOR-Operation mit einem Nachrichtenblock bereitsteht, wird das Little-Endian Format wieder zurückverwandelt [Ber05]. In der folgenden Abbildung 2.2 sieht man einen Block im initialen Zustand. Hier kann man außerdem nachvollziehen, wie die einzelnen Felder mit den Eingangswerten belegt werden.

"expa"	key	key	key
key	"nd 3"	nonce	nonce
pos.	pos.	"2-by"	key
key	key	key	"te k"

Abbildung 2.2: Der initiale Zustand eines Blocks [SKS22]. Die Felder bilden zeilenweise den Array von Index null bis 15.

Um einen Block zu erzeugen, werden zwei 32-Bit-Zahlen im Block miteinander mod 2^{32} addiert, eine XOR-Operation zwischen ihnen ausgeführt und 32-Bit-Rotationen nach links ausgeführt. All diese Operationen werden nach einem bestimmten Muster auf den Feldern des Arrays vollzogen, sodass man den Anfangszustand eines Blocks nicht mehr zurückverfolgen kann. Die genannten Operationen spiegeln sich in der Funktion $quarterround(y_0, y_1, y_2, y_3)$ wider, die folgendermaßen definiert ist, wobei y_i und z_i für beliebige Felder im Array (Block) stehen [Ber08] [Ber05].

$$\begin{aligned}
 z_1 &= y_1 \oplus ((y_0 + y_3) \lll 7); \\
 z_2 &= y_2 \oplus ((z_1 + y_0) \lll 9); \\
 z_3 &= y_3 \oplus ((z_2 + z_1) \lll 13); \\
 z_0 &= y_0 \oplus ((z_3 + z_2) \lll 18);
 \end{aligned} \tag{2.6}$$

Die Funktion $quarterround(y_0, y_1, y_2, y_3)$ wird in den Funktionen $rowround(y_0, \dots, y_{15})$ und $columnround(y_0, \dots, y_{15})$ jeweils viermal mit unterschiedlichen Parametern aufgerufen, sodass alle Felder im Block miteinander vermischt werden. Anschließend werden diese zwei Funktionen jeweils zehnmal nacheinander beginnend mit der $columnround$ Funktion aufgerufen. Das Ergebnis ist ein Block, auf dem mit dem jeweiligen Nachrichtenblock eine XOR-Operation durchgeführt werden kann. Um einen weiteren Block zu erzeugen, wird einfach der Positionszähler um eins erhöht und der initiale Zustand wieder hergestellt. Die Funktionen werden dann wieder wie im Text beschrieben aufgerufen [Ber05].

Wie man gut nachvollziehen kann, ist die $quarterround$ Funktion das zentrale Element für die Konstruktion des Ver- und Entschlüsselungsalgorithmus Salsa20. In dieser Funk-

²Die Werte 0x61707865, 0x3320646e, 0x79622d32 und 0x6b206574 wurden vom Erfinder festgelegt und ergeben den Text „expand 32-byte k“ [Ber05].

tion werden nur sehr einfache Operationen eingesetzt, weshalb sich dieser Algorithmus gut für den Mikrocontroller eignet.

2.4 Hamming-Code

Hamming-Codes wurden entwickelt, um 1-Bit-Fehler zu korrigieren und 2-Bit-Fehler zu erkennen. Um den Hamming-Code auf eine Nachricht anzuwenden, werden Paritätsbits mit gerader Parität an Bit-Positionen mit einer Zweierpotenz gesetzt. Es werden die Zweierpotenzen dafür verwendet, da diese immer nur ein Bit in ihrer jeweiligen Binärrepräsentation gesetzt haben. Die Paritätsbits schließen dann alle Bitpositionen der Nachricht ein, die ebenfalls ein gesetztes Bit dort haben. Das Paritätsbit an der Position $2^3 = 1000_2 = 8_{10}$ prüft nur die gemeinsame Parität an Positionen, an denen das Bit an der vierten Stelle gesetzt ist. Also zum Beispiel die Positionen neun, zehn und elf. Dies wird als eine Paritätsgruppe mit einem Paritätsbit bezeichnet. Das Paritätsbit wird so gesetzt, sodass eine gerade Parität innerhalb dieser Paritätsgruppe entsteht. Wenn ein Bit-Fehler innerhalb einer Nachricht auftaucht, werden die jeweiligen Paritätsgruppen überprüft. Das Ergebnis x der Überprüfung ist eine n lange Binärzahl, wobei n die Anzahl der Paritätsbits in der Nachricht ist. Wenn zum Beispiel die Parität für die Paritätsgruppen mit den Paritätsbits 2^3 und 2^1 ungerade ist, wird x zu 1010_2 . Alle anderen Paritäten sind hier weiterhin gerade und x repräsentiert die Position, an der der Fehler geschehen ist. Die Paritätsbits stehen an Zweierpotenzen, da diese in der Binärdarstellung immer ein gesetztes Bit haben. Diese bilden eine Paritätsgruppe, in der nur die Positionen enthalten sind, die ebenfalls an der gleichen Stelle ein gesetztes Bit besitzen. Dieses Beispiel kann man in den Tabellen 2.1 noch einmal genauer betrachten. Die Bitpositionen sind dabei zeilenweise von links nach rechts zu lesen, wobei die blauen Felder für die Zweierpotenzen mit den Paritätsbits stehen. Wie man daran sieht, wird die Position des Bitfehlers dadurch immer weiter eingeschränkt, sodass am Ende nur noch eine mögliche Position übrig bleibt. Sollte kein Fehler vorhanden sein oder der Fehler korrigiert werden, ist $x = 0000_2$. Um einen 2-Bit-Fehler zu erkennen, wird ein Paritätsbit an der Position 0 der Nachricht gesetzt. Dies entspricht den roten Feldern in den Tabellen 2.1. Diese gerade Parität gilt nun für die komplette Nachricht, sodass ein 2-Bit-Fehler vorliegt, wenn x einen Fehler anzeigt und die Parität über die komplette Nachricht trotzdem noch gerade ist [KU07] [Ham50].

	A	B	C	D
1	1	1	1	1
2	0	0	1	1
3	0	1	0	1
4	0	1	1	0

Korrekte Nachricht mit
Hamming-Code

	A	B	C	D
1	1	1	1	1
2	0	0	1	1
3	0	1	1	1
4	0	1	1	0

Fehlerhafte Nachricht mit
Hamming-Code an Position
C3

Tabelle 2.1: Beispielhafte Darstellung des Hamming-Codes.

Kapitel 3

Konzept und Vorgehensweise

In diesem Kapitel wird die Vorgehensweise für die kommende Implementierung vorgestellt. Außerdem werden Gründe aufgezeigt, warum manche Sachen auf eine bestimmte Weise umgesetzt wurden. Dabei geht es auch zu zeigen, was die Ziele und Vorhaben hinter den einzelnen Dingen sind.

MAGIC-Modus Für MAGIC hat bisher noch keine Implementierung existiert, weshalb dies zu Beginn auf einem normalen Computer mit x86-Prozessor umgesetzt wurde. Dabei wurde sich an der offiziellen Spezifikation von Michael Kounavis u. a. [Kou+20] und der Veröffentlichung von Lukas Lamster u. a. [Lam+24] orientiert. Später soll der MAGIC-Modus auf einem Mikrocontroller laufen können, weshalb die komplette Implementierung in C geschehen ist. Bevor man mit der Implementierung von MAGIC beginnen konnte, musste man einen Weg finden, Elemente vom Körper $GF(2^{128})$ darzustellen. In C ist dies nur über Umwege möglich. Die Elemente des Körpers kann man als 128-Bit große Zahlen auffassen. Dazu wird ein Array a mit vier 32-Bit-Feldern genutzt. Insbesondere hat man auch einzelne Operationen umgesetzt, sodass man mit den 128-Bit großen Zahlen auch sinnvoll arbeiten kann. Die 32-Bit-Felder passen sehr gut zum eingesetzten Mikrocontroller, da dieser 32-Bit-Register nutzt. Anschließend wurde die Arithmetik für den endlichen Erweiterungskörper $GF(2^{128})$ implementiert, welche mit dem beliebigen Array a arbeitet. Erst ab diesem Punkt konnte man den eigentlichen MAGIC-Modus in C implementieren.

Salsa20 Für die Ver- und Entschlüsselung wird Salsa20 genutzt, da bei diesem Verfahren nur einfache Operationen eingesetzt werden. Dies ist auf einem Mikrocontroller mit beschränkten Ressourcen sehr sinnvoll. Des Weiteren arbeitet Salsa20 und Advanced Encryption Standard (AES) intern mit 32-Bit-Werten, was gut zu den 32-Bit-Registern des Mikrocontrollers passt. Letztendlich wurde Salsa20 ausgewählt, da seine Implementierung im Vergleich zu AES sehr simpel und kompakt ist. Der Grund für eine eigene Umsetzung von Salsa20 war es, dass Abhängigkeiten zu anderen Bibliotheken und

anderen Implementierungen aufgrund des Mikrocontrollers vermieden werden sollten. Außerdem ist dieses Verfahren schneller als AES [Ber08]. Des Weiteren kann man durch Salsa20 leicht Zufallszahlen erzeugen. Für die Umsetzung wurde sich an der offiziellen Spezifikation von Daniel J. Bernstein [Ber05] orientiert. Um sicherzustellen, dass Salsa20 korrekt funktioniert, werden Testvektoren aus der offiziellen Spezifikation [Ber05] verwendet. Des Weiteren werden Known Answer Tests (KATs) zum Testen der kompletten Ver- und Entschlüsselung aus einem bestehenden Projekt [Boc12] benutzt.

Hamming-Code Da der MAGIC-Modus mit dem Hamming-Code in Bezug auf die Fehlerkorrektur verglichen werden muss, wird dieser ebenfalls selbst implementiert. Ansonsten ist es schwer zu schlussfolgern, wie gut die Fehlerkorrektur von MAGIC auf einem Mikrocontroller funktioniert. Der Hamming-Code wurde selbst implementiert, um Abhängigkeiten auf dem Mikrocontroller zu vermeiden. Dazu nutzt der Hamming-Code ebenfalls den Token und die verschlüsselten Blöcke, welche durch MAGIC erzeugt werden. Dadurch wird sichergestellt, dass nur die Fehlerkorrektur miteinander verglichen wird. Insbesondere muss dadurch kein weiteres Verfahren zur Erzeugung der verschlüsselten Blöcke und des Tokens implementiert werden. Allerdings kann ein beliebiger Hashwert für die Tokengenerierung genutzt werden, da der Token nicht für die Fehlerkorrektur relevant ist. Der Token wird gebraucht, um die Authentizität einer Nachricht sicherzustellen. Eigentlich braucht der Hamming-Code somit diesen Token nicht. Um allerdings die Fehlerkorrektur der beiden Verfahren sinnvoll vergleichen zu können, wird dieser Token auch beim Hamming-Code genutzt. Außerdem kann man eine bessere Aussage im Kontext der Fehlerkorrektur treffen, wie MAGIC im Vergleich zum Hamming-Code abschneidet. Es kann sein, dass das eine oder andere Verfahren nur in bestimmten Punkten besser oder schlechter ist. Den Hamming-Code kann man einerseits pro verschlüsselten Block oder über alle verschlüsselten Blöcke hinweg implementieren. Beides hat seine Vor- und Nachteile. Wenn man den Hamming-Code pro Block umsetzt, kann man mehrere fehlerhafte Blöcke einer Nachricht korrigieren, bzw. die Fehler erkennen. Jedoch braucht dies mehr Speicherplatz, da acht Paritätsbits pro 128-Bit-Block anfallen. Beim anderen Weg braucht man weniger Speicherplatz, da nur $1 + \log_2(128 * n)$ viele Paritätsbits für die komplette Nachricht benötigt werden. Hierbei steht n für die Blockanzahl. Jedoch kann man keine Fehler mehr korrigieren, wenn mehrere Blöcke von einem Bit-Fehler betroffen sind. Das ist allerdings beim MAGIC-Modus auch so. Es wurden beide Verfahren des Hamming-Codes implementiert, um diese jeweils mit dem MAGIC-Modus miteinander zu vergleichen. Dadurch kann man eine noch bessere Einordnung der Evaluierungsergebnisse bekommen.

Tests Des Weiteren wurden alle hier genannten Verfahren mithilfe von Unittests auf ihre Richtigkeit überprüft. Hier machen nur Unittests Sinn, da keine Anbindungen zu anderen Systemen wie beispielsweise Datenbanken notwendig sind. Alle Unittests können auch auf dem Mikrocontroller ausgeführt werden. Um die Tests schnell zu starten

und alle Teile des Programms schnell zu kompilieren, wurde ein Makefile angelegt. Dabei gibt es eines für die Ausführung auf dem normalen Computer und eines für die Ausführung auf dem Mikrocontroller.

Mikrocontroller Letztendlich muss die MAGIC-Implementierung und der Hamming-Code auf dem Mikrocontroller funktionieren und dort evaluiert werden. Der Mikrocontroller besteht aus einem Cortex-M3-Prozessor mit externen SRAM. Um den Mikrocontroller einzurichten und zu starten, werden Teile eines bestehenden Projekts [Inc22] genutzt. Dort befindet sich das genannte Makefile für den Mikrocontroller. Wenn man dieses auf dem normalen Computer ausführt, wird eine Datei erstellt, die auf dem Mikrocontroller ausgeführt werden kann. Um den Code auf dem Mikrocontroller zu bringen, wird das Joint Test Action Group (JTAG)-Protokoll¹ genutzt. Mithilfe von JTAG und OpenOCD kann man das Programm auf dem Chip ausführen und debuggen. Dazu muss man folgendes Kommando ausführen.

```
sudo openocd -f [cfg-Datei]
```

Dies öffnet einen Port für gdb-Verbindungen zum angeschlossenen Mikrocontroller. Anschließend führt man in einem weiteren Terminal folgende Befehle aus, um die erstellte Datei auf dem Mikrocontroller auszuführen.

```
arm-none-eabi-gdb ./[Dateiname].elf
> target extended-remote :3333
> load
> run/step/next/continue/break/watch
```

Es wird dafür eine spezielle gdb-Ausführung für ARM-Prozessoren genutzt. Mithilfe der zwei weiteren Befehle wird der Mikrocontroller programmiert und die vorher angegebene Datei geladen. Anschließend kann man die bekannten gdb-Befehle nutzen, um das Programm direkt auf dem Mikrocontroller zu debuggen.

Evaluierungsergebnisse Um die Evaluierungsergebnisse vom Mikrocontroller festzuhalten, wird gdb zu Hilfe genommen. Dazu wurde der Befehl `dprintf` genutzt. Dieser Befehl kombiniert einen Breakpoint mit einem `printf` aus C. Jedoch stoppt das Programm dort dann nicht, sondern erzeugt nur eine Ausgabe. Der komplette Befehl steht dann pro Evaluierung in einer externen Datei, welche mit `source [Dateipfad]` eingelesen werden kann. Mit den logging-Optionen in gdb kann man den erzeugten Output durch `dprintf` in eine weitere Datei umlenken, sodass man die Evaluierungsergebnisse dort dann jederzeit betrachten kann. Genauer gesagt werden hier die Anzahl der Takte

¹Das JTAG-Protokoll ermöglicht das Programmieren, Debuggen und Testen von ICs, Prozessoren und FPGAs direkt in der Schaltung [Mic].

für die jeweiligen Operationen festgehalten. Die Auswertung der Takte folgt in Kapitel 5. Ein kompletter Befehl sieht zum Beispiel folgendermaßen aus [Bue21].

```
dprintf main_cw521.c:94, "Number of blocks: %i\n", i
```

In diesem Beispiel wird die Blockanzahl in Zeile 94 ausgegeben und dann mit den gesetzten logging-Optionen in eine Datei geschrieben.

Kapitel 4

Implementierung

Im folgenden Kapitel wird auf die Implementierung und technische Umsetzung der Problemstellung eingegangen. Vor allem werden Entwurfsentscheidungen vorgestellt und gezeigt, wie bestimmte Dinge umgesetzt wurden.

Die Umsetzung ist in der Programmiersprache C erfolgt, da die komplette Implementierung auf einem Mikrocontroller lauffähig sein soll. Außerdem ist auf diesem die Performance nicht sehr hoch und der Speicherplatz ist ebenfalls begrenzt. Beide Punkte sprechen für C, da man damit die Nutzung der Ressourcen sehr gut steuern kann. Insbesondere ist die Entwicklung durch den Mikrocontroller sehr hardwarenah, was ebenfalls für C spricht.

4.1 Programmaufbau

Das Projekt ist in verschiedene Ordner und Dateien aufgeteilt. Im Hauptordner befinden sich Dateien aus dem ChipSHOUTER-ballisticgel Projekt [Inc22]. Es werden dabei nur Dateien aus dem Ordner `firmware/cw521` von dem genannten Projekt verwendet. Hiermit hat man das Board mit dem Mikrocontroller schnell eingerichtet. Im Hauptordner des Quellcodes für die Bachelorarbeit befindet sich eine Konfigurationsdatei für die JTAG-Anbindung mit OpenOCD. Diese wurde selbst erstellt und war kein Teil des genannten Projekts. Ebenfalls befindet sich dort die Datei `main_cw521.c`, in der die implementierten Funktionen aus `magic_bachelorarbeit` ausgeführt werden. Hier werden außerdem die Takte für die Evaluierung gemessen. Die restlichen Dateien im Hauptordner sind für die Codeausführung auf dem Mikrocontroller verantwortlich und sind nicht weiter interessant für diese Arbeit. In der Datei `cw521.elf` steht der kompilierte Maschinencode für den Mikrocontroller.

Im Ordner `magic_bachelorarbeit` des Quellcodes für die Bachelorarbeit befindet sich die selbst erstellte Implementierung. In der Abbildung 4.1 sieht man die Beziehung zwischen den einzelnen Programmteilen aus dem Unterordner `src`. Dieses Diagramm hat

sich am Unified Modeling Language (UML)-Klassendigramm orientiert, wobei hier die Klassen durch die Dateien, bzw. Programmteile ersetzt wurden. Es sind nicht alle Dateien in der Abbildung 4.1 dargestellt. Es fehlen die Dateien `main.c` und `examples.c` in dieser Abbildung, da diese nur für die Codeausführung verantwortlich sind. In `main.c` ist die `main` Funktion enthalten und in `examples.c` befinden sich Beispiele für die komplette Ausführung von MAGIC und des Hamming-Codes. Dort werden zu Beginn ein zufälliger Schlüssel und eine Nonce erzeugt. Danach kommt die Erzeugung der verschlüsselten Blöcke und anschließend findet die Fehlerkorrektur von einem Bitfehler statt. Ein Pfeil im Diagramm 4.1 deutet hierbei darauf hin, dass Funktionen von der einen Datei in der anderen Datei genutzt werden. Zum Beispiel werden Funktionen aus `bignum` in `galois` genutzt.

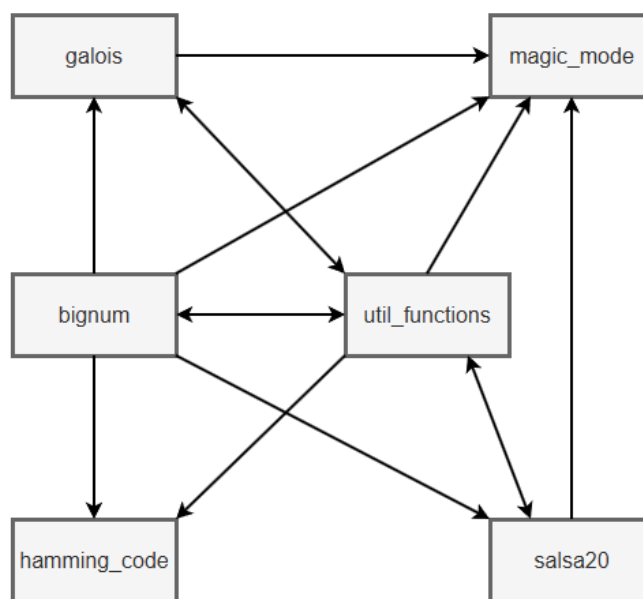


Abbildung 4.1: Der grundlegende Aufbau der MAGIC und Hamming-Code Implementierung.

4.2 Implementierung von 128-Bit großen Zahlen

Der MAGIC-Modus kann theoretisch mit beliebig großen Zahlen, bzw. Blockgrößen arbeiten. Jedoch ist ein konkretes Beispiel für eine Blockgröße von 128-Bit mit dem irreduziblen Polynom $x^{128} + x^7 + x^2 + x + 1$ definiert worden [Kou+20]. Daran wurde sich orientiert, da 128-Bit eine gute Sicherheit bietet und 64-Bit dagegen nicht mehr. Außerdem ist 128 ein Vielfaches von 32, was sehr gut zu den 32-Bit großen Registern im eingesetzten Mikrocontroller passt. Es wird ein Array mit vier 32-Bit großen Feldern eingesetzt, um die 128-Bit großen Zahlen darzustellen. Die Implementierung davon ist in der Datei `bignum.c` zu finden.

Zunächst wird das Array, wie in Quellcode 4.1 zu sehen ist, innerhalb eines `struct` definiert. Das wurde zu `bignum` umbenannt, um es einfacher zu referenzieren. Das Array

wird mit dem Datentyp `uint32_t` und einer festen Anzahl von vier Feldern definiert, wodurch eine 128-Bit-Zahl dargestellt werden kann. Außerdem wird der `unsigned` Datentyp genutzt, da es aufgrund des endlichen Erweiterungskörpers nicht notwendig ist, negative Zahlen zu nutzen.

```
1 | typedef struct {  
2 |     uint32_t chunks[4];  
3 | } bignum;
```

Quellcode 4.1: Die Darstellung einer 128-Bit-Zahl.

Das Array ist dabei innerhalb eines `struct` definiert, da man ansonsten nicht direkt erkennen kann, ob es sich um ein Array handelt. Dies ist vor allem problematisch, wenn man ihn einer Funktion übergeben möchte. Die Alternative würde nämlich folgendermaßen aussehen.

```
typedef uint32_t bignum[4];
```

Also ist es besser, ihn in einem `struct` zu definieren, da man dadurch besser erkennen kann, dass es sich um einen `uint32_t` Array handelt. Dadurch wird es einfacher, ein Array vom Datentyp `bignum` zu erstellen.

Zu Beginn der Funktionen in `bignum.c` werden oft die Eingangsparameter kopiert, um falsche Resultate zu vermeiden. Wenn ein Eingangsparameter beim Funktionsaufruf ebenfalls den Rückgabewert bekommt, könnte es ansonsten zu falschen Ergebnissen kommen. Daher werden diese kopiert, um sie in den Funktionen nicht direkt zu verändern.

Initialisierung Um den Datentyp `bignum` zu initialisieren, muss das Array `chunks` aus dem `struct` in Quellcode 4.1 mit Zahlen vom Datentyp `uint32_t` befüllt werden. Dazu gibt es die Funktion `init_bignum`, die als Eingangsparameter ein Array mit vier 32-Bit-Feldern besitzt. Die 128-Bit-Zahl wird auf diese Felder beginnend mit dem höchstwertige Bit (MSB) aufgeteilt, und dann in `chunks` kopiert, sodass der Datentyp `bignum` entsteht und zurückgegeben wird. Somit steht das MSB in `chunks[0]` und das niedrigstwertige Bit (LSB) in `chunks[3]`. Der einfache Grund dafür ist, dass es den restlichen Quellcode und die Implementierung intuitiver macht.

XOR-Operation Des Weiteren ist es möglich, eine XOR-Operation zwischen zwei initialisierten `bignum` auszuführen. Diese hat eine hohe Relevanz, um die Arithmetik in den endlichen Körpern zu implementieren. Dazu findet eine XOR-Operation zwischen den jeweiligen Feldern des Arrays statt. Jedes Feld in `chunks` kann dabei getrennt von den anderen betrachtet werden, da bei einer XOR-Operation keine Überträge entstehen. Die XOR-Operation ist nur eine Addition mod 2, bei der bekannterweise keine Überträge entstehen.

Bit-Shifting Außerdem ist das Bit-Shifting nach links und rechts um jeweils ein Bit möglich, da dies bei der Arithmetik in endlichen Körpern benötigt wird. Hier kann man die einzelnen Array-Felder nicht getrennt voneinander betrachten, da beispielsweise das MSB aus `chunks[3]` zum LSB in `chunks[2]` bei einem Left-Shift um eins wird. Die Realisierung eines Left-Shifts um eins kann man sich in Quellcode 4.2 anschauen. In Zeile sieben wird das letzte Feld isoliert betrachtet, da kein vorheriges Feld Auswirkungen auf das letzte Feld bei einem Left-Shift haben kann. Bei allen anderen Feldern muss man die nachfolgenden Felder jedoch beachten. Durch die OR-Operation in Zeile zehn wird bestimmt, ob das MSB des nachfolgenden Feldes Auswirkungen auf das LSB des resultierenden Feldes hat. Das Resultat ist ein Feld, das unter der Beachtung des nachfolgenden Blocks nach links verschoben wurde. Dies funktioniert, weil durch den Right-Shift um 31-Bit nur noch das MSB des nachfolgenden Feldes übrig bleibt, welches das aktuelle Feld beeinflussen könnte. Durch OR wird anschließend geprüft, ob das LSB des aktuellen Feldes angepasst werden muss. Das MSB aus `chunks[0]` geht bei diesem Algorithmus verloren, was so gewollt ist. In Abschnitt 4.3 wird deutlich, warum dies der Fall ist. Im Allgemeinen kann man dazu sagen, dass die Zahlen im endlichen Körper bleiben müssen.

```

1  bignum shift_left_by_one_bignum(bignum n) {
2      bignum tmp_n = copy_bignum(n);
3      bignum result;
4
5      for (int i = 0; i < 4; i++) {
6          if (i == 3) {
7              result.chunks[i] = tmp_n.chunks[i] << 1;
8          }
9          else {
10             result.chunks[i] = (tmp_n.chunks[i] << 1) | (tmp_n.chunks[i
11                 +1] >> 31);
12         }
13     }
14     return result;
15 }
```

Quellcode 4.2: Die Umsetzung einer Shift-Left-Operation für `bignum`.

Der Right-Shift um ein Bit funktioniert prinzipiell wie der Left-Shift aus dem Quellcode 4.2. Die Unterschiede liegen darin, dass `chunks[0]` isoliert betrachtet wird und die Bit-Shifts umgekehrt werden. Die Logik bleibt die Gleiche wie beim Left-Shift.

Operationen wie zum Beispiel die Addition und Multiplikation wurden nicht implementiert, da diese nicht benötigt werden. Nur Funktionen, die auch wirklich gebraucht werden, sind auch implementiert worden. Die XOR und Shifting-Operationen gehören hierbei zu den komplexeren und wichtigen Funktionen. Alle restlichen Funktionen sind trivial und in sehr wenigen Codezeilen umgesetzt worden, weshalb diese hier nicht weiter ausgeführt werden. Im Allgemeinen kann man sagen, dass die Funktionen in `bignum` existieren, um den Code für die Arithmetik in Körpern in Abschnitt 4.3 zu vereinfachen und ein einheitliches Konzept für die Darstellung von großen Zahlen zu haben. Deshalb

gibt es keine weiteren Operationen, um das Rechnen mit `bignum` komplett abzudecken.

4.3 Arithmetik in endlichen Erweiterungskörpern

Der MAGIC-Modus arbeitet auf dem endlichen Erweiterungskörper $GF(2^{128})$ mit dem irreduziblen Polynom $x^{128} + x^7 + x^2 + x + 1$ [Kou+20]. Aus diesem Grund muss die Arithmetik auf diesem Körper ebenfalls implementiert werden, um MAGIC korrekt umzusetzen. Konkret braucht man in dieser Arithmetik die Operationen Addition, Multiplikation und das Finden des inversen Elements bezüglich der Multiplikation.

Addition Zunächst muss man zwei Zahlen/Blöcke in MAGIC addieren können. Dazu reicht eine XOR-Operation zwischen den zwei Zahlen aus [PP16]. Das Ergebnis ist die Addition in einem endlichen Erweiterungskörper, wie man schon in Kapitel 2 erfahren konnte. Für die XOR-Operation wird die Funktion aus `bignum.c` verwendet.

Multiplikation Die Multiplikation ist nicht so trivial wie die Addition. Würde man eine klassische Polynommultiplikation mit anschließender Reduktion durch das irreduzible Polynom durchführen, dauert dies mit 128-Bit großen Zahlen viel zu lange [VK07]. Deswegen wurde die Multiplikation mithilfe der „Russischen Bauernmultiplikation“ umgesetzt, welche in [Cev18] beschrieben ist. Die Implementierung ist im folgenden Quellcode 4.3 zu sehen und macht sich die Binärdarstellung der Zahlen zunutze. Zunächst wird der Operand (Parameter) a immer wieder halbiert und der andere Operand (Parameter) b immer wieder verdoppelt. In Zeile 13 wird a halbiert und in Zeile 16 wird b verdoppelt, wofür einfache Shifting-Operationen ausreichen. Das Ganze geht so lange, bis alle Bits einer Zahl abgearbeitet wurden, was man in Zeile acht sieht. In der Theorie [Cev18] wird a so lange halbiert, bis es eins ist. Sollte a bei der Halbierung ungerade sein, wird das Ergebnis auf die nächste natürliche Zahl abgerundet, was durch das Bit-Shifting im Quellcode 4.3 automatisch geschieht. Währenddessen werden die b Werte miteinander addiert, wenn a zur gleichen Zeit ungerade ist, was in den Zeilen neun bis elf zu sehen ist. In der Implementierung reicht dazu die XOR-Operation in Zeile zehn aus, um zwei Zahlen zu addieren, da man sich immer noch im Körper $GF(2^{128})$ befindet. Die Summe von b Werten ist das Ergebnis der Multiplikation. Sollte b während der Durchführung außerhalb des Erweiterungskörpers liegen, wird $b = b \bmod (x^{128} + x^7 + x^2 + x + 1)$ gerechnet. Dies kann man ebenfalls mit der Operation $b = b \oplus (x^7 + x^2 + x + 1)$ erreichen, was in [Kak25] auf den Seiten 22 bis 24 beschrieben ist. Im Quellcode 4.3 wird dazu zunächst in Zeile 15 geprüft, ob das MSB an Position 128 gesetzt ist. Wenn das der Fall ist, liegt b nicht mehr innerhalb des Körpers nach der Verdopplung. Durch die XOR-Operation in Zeile 19 wird b wieder innerhalb des Körpers gebracht. Deshalb ist es auch nicht notwendig, das MSB bei einem Left-Shift um eins weiter zu beachten.

```

1  bignum mult(bignum a, bignum b) {
2      bignum tmp_a = copy_bignum(a);
3      bignum tmp_b = copy_bignum(b);
4
5      bignum result = init_bignum_to_zero();
6      bool inside_galois_field;
7
8      while (is_bignum_not_zero(tmp_a) && is_bignum_not_zero(tmp_b)) {
9          if (is_bignum_odd(tmp_a)) {
10             result = xor_bignum(result, tmp_b);
11         }
12
13         tmp_a = shift_right_by_one_bignum(tmp_a);
14
15         inside_galois_field =
16             is_bignum_inside_galois_field_after_left_shift(tmp_b);
17         tmp_b = shift_left_by_one_bignum(tmp_b);
18
19         if (!inside_galois_field) {
20             tmp_b = xor_bignum(tmp_b, IRREDUCIBLE_POLYNOMIAL_128);
21         }
22     }
23     return result;
24 }

```

Quellcode 4.3: Die Multiplikation in $GF(2^{128})$.

Das irreduzible Polynom `IRREDUCIBLE_POLYNOMIAL_128` aus dem Quellcode 4.3 wurde zu Beginn der Datei `galois.c` als `bignum` Konstante definiert. In der Darstellung als Bitstring braucht das Polynom 129 Bits und passt nicht in den Datentyp `bignum`. Deshalb wird das MSB des irreduziblen Polynoms nicht für die `bignum` Darstellung beachtet. Da man das Polynom nur braucht, um eine zu große Zahl wieder in den Körper $GF(2^{128})$ zu bringen, kann man das MSB in diesem Fall einfach vernachlässigen, wie man in [Kak25] sehen kann.

Inverses Element Um das inverse Element bezüglich der Multiplikation zu finden, wird der kleine Satz von Fermat angewendet, wodurch folgender Ausdruck entsteht $n^{2^{128}-2} = n^{-1}$. Es kann der kleine Satz von Fermat verwendet werden, da Modulo mit einem irreduziblen Polynom gerechnet wird. Dieses Polynom verhält sich wie eine Primzahl, was beim kleinen Satz von Fermat notwendig ist. Im genannten Ausdruck ist n die Zahl, von der das inverse Element berechnet werden soll. Um diese große Potenz zu berechnen, wird ein Verfahren angewendet, welches in [VK07] auf Seite fünf beschrieben wird. Dazu muss man den Exponenten als Binärzahl sehen. Es sind mehrere Multiplikationen von Potenzen, deren Exponent eine Zweierpotenz ist. Zum Beispiel wird x^{45} zu $x^{32} \cdot x^8 \cdot x^4 \cdot x$, wodurch man nur noch x^2 , x^4 , x^8 und x^{32} berechnen muss. Im Quellcode 4.4 sieht man dieses Verfahren. In Zeile zwölf wird `base` immer wieder quadriert. Wenn das aktuelle Bit des Exponenten gesetzt ist, wird in Zeile acht bis zehn die entsprechende `base` zum Resultat dazu multipliziert. Ist somit das vierte Bit des Exponenten gesetzt, findet eine Multiplikation zwischen `base4` und dem Resultat statt. Durch den Right-Shift in Zeile 14, werden durch alle Bits des Exponenten iteriert.

```

1 | bignum power(bignum base, bignum exponent) {
2 |     bignum base_tmp = copy_bignum(base);
3 |     bignum exponent_tmp = copy_bignum(exponent);
4 |
5 |     bignum result = init_bignum_to_one();
6 |
7 |     while (is_bignum_not_zero(exponent_tmp)) {
8 |         if (is_bignum_odd(exponent_tmp)) {
9 |             result = mult(result, base_tmp);
10 |        }
11 |
12 |        base_tmp = mult(base_tmp, base_tmp);
13 |
14 |        exponent_tmp = shift_right_by_one_bignum(exponent_tmp);
15 |    }
16 |
17 |    return result;
18 | }

```

Quellcode 4.4: Eine große Potenz berechnen, um das inverse Element zu finden.

4.4 Umsetzung der Ver- und Entschlüsselung durch Salsa20

Für die Umsetzung von Salsa20 wurde sich an der offiziellen Spezifikation von Daniel J. Bernstein [Ber05] orientiert. Wie Salsa20 genau funktioniert, wurde bereits in den Grundlagen aus Kapitel 2 erläutert. Demzufolge werden manche Funktionen nicht noch einmal erläutert.

Zunächst musste die quarterround Funktion umgesetzt werden. Dazu braucht es eine Shift-Left-Rotate Operation. In C gibt es dafür keinen Operator, weshalb man diese Operation selbst implementieren musste. Im folgenden Quellcode 4.5 sieht man die Implementierung davon. Zunächst wird die 32-Bit-Zahl `value` um die gewünschte Anzahl von Left-Shifts verschoben. Um die verloren gegangenen Bits wieder auf die rechte Seite hinzuzufügen, findet eine OR-Operation mit diesen verlorenen Bits und der verschobenen Zahl statt.

```

1 | uint32_t shift_left_and_rotate(uint32_t value, int number_of_shifts) {
2 |     return (value << number_of_shifts) | (value >> (32 -
3 |         number_of_shifts));

```

Quellcode 4.5: Die Shift-Left-Rotate Operation für Salsa20.

Wie in der Spezifikation [Ber05] dargestellt, werden nun die Funktionen `rowround`, `columnround` und `doubleround` umgesetzt. Als Input dieser Funktionen dient jeweils ein 32-Bit-Array mit 16 Feldern. Dieses Array repräsentiert den internen Block/Zustand (state) von Salsa20 in folgender Form.

```
uint32_t state[16]
```

Die Funktion `littleendian` wurde ebenfalls implementiert, da Salsa20 dadurch effi-

zienter die Operationen ausführen kann [Ber08]. Als Input dieser Funktion dient eine 32-Bit-Zahl, was einem Feld von `state` entspricht. Die `salsa20_hash`-Funktion ist dazu da, um den internen Zustand in das Little-Endian-Format zu bringen und anschließend zehnmal die `doubleround`-Funktion auszuführen. Danach wird der `state` wieder in das originale Format gebracht, indem die `littleendian`-Funktion erneut aufgerufen wird. Durch das zehnmahlige Aufrufen der Funktion `doubleround` wird `state` vermischt. Als Input-Parameter wird zweimal der `state` genutzt. Einmal als Input und das andere mal als Output. Als Nächstes wird durch `salsa20_expansion` der Anfangszustand initialisiert, der in 2.2 zu sehen ist. Nach dieser Initialisierung wird `salsa20_hash` in dieser Funktion aufgerufen.

Abschließend findet die eigentliche Ver- und Entschlüsselung mithilfe der Funktion `salsa20_encryption_decryption` statt. Als Input wird ein 256-Bit-Schlüssel, eine 64-Bit-Nonce, eine zu verschlüsselnde Nachricht x als 32-Bit-Array und die Anzahl der 32-Bit-Felder der zu verschlüsselten Nachricht genutzt. Außerdem wird ein weiterer Input-Parameter y genutzt, um die verschlüsselte Nachricht zu speichern. Dazu wird ebenfalls ein 32-Bit-Array verwendet. Für x und y wurde ein einfaches Array vom Typ `uint32_t` genutzt und nicht ein Array vom Typ `bignum`, da dadurch Nachrichten beliebiger Länge verschlüsselt werden können. Außerdem passt dies besser zur Spezifikation [Ber05] von Bernstein. Zunächst wird die Position beginnend von null in `state` gesetzt und die Funktion `salsa20_expansion` aufgerufen, um `state` zu initialisieren und zu vermischen. Anschließend wird über die komplette Nachricht iteriert und es findet eine XOR-Operation zwischen einem Feld von x und einem Feld von `state` statt. Sobald ein kompletter `state` genutzt und somit alle seine 16 32-Bit-Felder abgearbeitet wurden, wird wieder ein neuer erzeugt. Dazu wird die Position inkrementiert und `salsa20_expansion` erneut aufgerufen. Den beschriebenen Vorgang kann man sich im folgenden Quellcode 4.6 nochmals verdeutlichen. Zum Aktualisieren der Position wird eine Funktion genutzt, um die 64-Bit große Position in die zwei passenden 32-Bit-Felder des Arrays zu bringen. Insbesondere ist anzumerken, dass die Entschlüsselung genauso funktioniert. Dazu werden die Rollen von x und y vertauscht. In x befinden sich die verschlüsselten Blöcke und in y wird sich der Klartext befinden.

```

1 // iterates over the message
2 for (int i = 0; i < message_length; i++) {
3     // a new state every 16 fields (a state has 16 array fields)
4     if (i % 16 == 0) {
5         position = update_position(position, nonce_with_position, i);
6         salsa20_expansion(state_in, state_out, key, nonce_with_position
7     );
8     }
9     y[i] = x[i] ^ state_out[i % 16];
10 }
```

Quellcode 4.6: Das Prinzip der Ver- und Entschlüsselung von Salsa20.

Um von einem String zu verschlüsselten `bignum`-Blöcken oder andersherum zu kommen, gibt es zwei Funktionen in `salsa20.c`. Diese nutzen beide die beschriebene Funk-

tion `salsa20_encryption_decryption`. Neben der eigentlichen Ver- oder Entschlüsselung finden in diesen Funktion Konvertierungen statt, um die Ver- und Entschlüsselungsfunktion richtig nutzen zu können. Danach finden nochmal Konvertierungen statt, um die richtigen Formate zurückzugeben. Diese Funktionen wurden in großen Teilen nur dafür umgesetzt, um mit dem MAGIC-Modus und Hamming-Code einfacher zu arbeiten. Die zwei Funktionen befinden sich in `salsa20.c`, da diese von MAGIC und vom Hamming-Code verwendet werden.

4.5 Realisierung des MAGIC-Modus

Um den MAGIC-Modus zu realisieren, wird die Arithmetik in den endlichen Erweiterungskörpern verwendet, den erstellten Datentyp `bignum` und `Salsa20`. In diesem Abschnitt werden Bezeichnungen genutzt, die bereits in Abschnitt 2.2 eingeführt worden sind.

Zunächst muss man die verschlüsselten Blöcke $C_1, \dots, C_n$ erzeugen. Dazu wird die Funktion aus `salsa20.c` verwendet, die einen String in die verschlüsselten `bignum`-Blöcke umwandelt. Das zentrale Element des MAGIC-Modus ist das Finden eines passenden Hashwertes H . Laut Lamster u. a. [Lam+24] muss man dazu die Gleichung 2.2 verwenden und damit einen zufälligen Hashwert H überprüfen. Bei dieser Arbeit ist dies ebenfalls so geschehen.

Um einen zufälligen Hashwert H zu finden, wird Salsa20 für die Generierung von Zufallszahlen verwendet. Es gibt zwar die Funktion `rand` in der C-Standardbibliothek, jedoch ist diese nicht sinnvoll, um qualitativ gute Zufallszahlen zu erzeugen [Asp14]. Des Weiteren könnte man die Dateien `/dev/random` oder `/dev/urandom` auf Linux nutzen [Asp14]. Da dies jedoch auf einem Mikrocontroller laufen soll, kommen diese Dateien nicht infrage. Des Weiteren kann man mit einer eigenen Implementierung durch Salsa20 direkt kryptografisch sichere Werte vom Typ `bignum` erzeugen. Es müssen dann keine weiteren Konvertierungen stattfinden. Ebenfalls kann man damit einen zufälligen Schlüssel und Nonce für Salsa20 erzeugen. Die Zufallsfunktion kann man im folgenden Quellcode 4.7 betrachten. Zunächst wird dort ein Schlüssel, eine Nonce und eine Nachricht mit dem Seed initialisiert. Diese Werte werden anschließend genutzt, um einen `bignum`-Wert, einen Schlüssel und eine Nonce zu erzeugen. In Zeile 20 und 23 wird die Nonce erhöht, da dieser Wert nur einmal genutzt werden sollte. Die Berechnung `seed+i` bei den Initialisierungen hat den Nutzen, um noch mehr Variation in die Zufallsfunktion zu bringen. Dies ist aber nicht zwingend erforderlich.

```

1 | bignum random_bignum_key_nonce(uint32_t key_res[8], uint32_t nonce_res
2 |     [2], uint32_t seed) {
3 |     uint32_t message_in[BLOCKSIZE];
4 |     uint32_t key[8];
5 |     uint32_t nonce[2];

```

```

6   for(int i = 0; i < 8; i++) {
7       key[i] = seed+i;
8   }
9   for(int i = 0; i < 2; i++) {
10      nonce[i] = seed+i;
11  }
12  for (int i = 0; i < 4; i++) {
13      message_in[i] = seed+i;
14  }
15
16  uint32_t message_out_bignum[4];
17  salsa20_encryption_decryption(key, nonce, message_in,
18      message_out_bignum, 4);
19  bignum random_bignum = init_bignum(message_out_bignum);
20
21  nonce[0] = nonce[0]+1;
22  salsa20_encryption_decryption(key, nonce, key, key_res, 8);
23
24  nonce[1] = nonce[1]+1;
25  salsa20_encryption_decryption(key, nonce, nonce, nonce_res, 2);
26
27  return random_bignum;
28 }

```

Quellcode 4.7: Zufällige Werte mit Salsa20 erzeugen.

Um einen zufälligen Hashwert zu validieren, muss man im nächsten Schritt alle möglichen Fehlervektoren e unterhalb eines bestimmten Schwellenwertes nutzen. Einen Fehlervektor für den MAGIC-Modus in $GF(2^{128})$ kann man als einen 128-Bitstring auffassen. Wenn kein Fehler vorliegt, ist dieser Fehlervektor an allen Stellen null. Wenn zum Beispiel ein Fehler an Bit 43 vorliegt, ist dieses Bit im Fehlervektor eine eins. Um alle Fehlervektoren mit einem bestimmten Schwellenwert T_{th} zu finden, muss man alle möglichen Kombinationen mit einer Anzahl von T_{th} Elementen aus den Werten von null bis 127 finden. Die Werte stehen für die möglichen Bitpositionen mit einem Fehler. Somit ist dies aus dem Bereich der Kombinatorik eine geordnete Stichprobe ohne Zurücklegen. Um alle Kombinationen dieser Fehlervektoren zu finden, wurde sich am Forward-Backward Algorithmus auf folgender Website [HMK18] orientiert. Dieser Algorithmus wird im Quellcode 4.8 genutzt, um die Hashwerte mit allen möglichen Fehlervektoren zu überprüfen. Dieser ist in der Datei `util_functions.c` zu finden. Hierbei ist zu Beginn `index_combinations` und `index_bit_indices` gleich null. Die allgemeine Idee dahinter ist, dass über die möglichen Bitpositionen `index_bit_indices` iteriert wird und die Felder im `combination`-Array für eine aktuelle Kombination aktualisiert werden. Der `combination`-Array umfasst T_{th} viele Elemente. Wenn eine Kombination für einen Fehlervektor ermittelt wurde, wird der Hashwert damit überprüft. Sollte die Überprüfung erfolgreich sein, wird `true` zurückgegeben. Ansonsten wird `false` ohne weitere Überprüfungen zurückgegeben. Der Name des Algorithmus kommt daher, dass man immer wieder nach vorn und zurück über alle Bitindexe von null bis 127 und über das `combination`-Array iteriert. Die Indexe von null bis 127 werden dabei nicht in einem Array gespeichert, da die Werte gleich dem Index im Array wären. Zum Beispiel würde dann an Index null die null stehen und an Index 127 die 127, wodurch es keinen Unterschied macht, ob man über die Elemente oder nur die Indexe iteriert.

```

1  while (index_combinations >= 0) {
2      // forward step as long as both indices does not exceed array
      lengths
3      if (index_bit_indices < 128 && index_combinations < threshold) {
4          // set value in the combination array
5          combination[index_combinations] = index_bit_indices;
6          // if combination array is complete do the calculations
7          if (index_combinations == threshold-1) {
8              bignum error_vector = polynom_to_bignum(combination,
9                  threshold);
10             // check hash-key
11             for (int i = 0; i < number_of_blocks; i++) {
12                 if (hamming_weight(mult(error_vector, hash_key_inverse)
13                     ) <= threshold || hamming_weight(mult(error_vector,
14                         hash_key)) <= threshold) {
15                     return false;
16                 }
17                 hash_key = mult(hash_key, hash_key_copy);
18                 hash_key_inverse = mult(hash_key_inverse,
19                     hash_key_inverse_copy);
20             }
21             // move forward in the bit indices
22             index_bit_indices++;
23         }
24         // select next bit index
25         else {
26             index_bit_indices = combination[index_combinations]+1;
27             index_combinations++;
28         }
29     }
30     // backward step
31     else {
32         // gets negative when the last combination was found -> all
33         // elements in combination array become 127
34         index_combinations--;
35         if (index_combinations >= 0) {
36             index_bit_indices = combination[index_combinations]+1;
37         }
38     }
39 }

```

Quellcode 4.8: Alle Kombinationen unter einem Schwellenwert von Fehlervektoren finden und damit den Hashwert überprüfen.

In `magic_mode.c` wird die Funktion mit dem Quellcode 4.8 jeweils mit einem bestimmten Schwellenwert aufgerufen. Um zum Beispiel einen Hashwert mit dem Schwellenwert drei zu finden, wird mithilfe einer Schleife über die Schwellenwerte von eins bis drei iteriert. Für jeden einzelnen Schwellenwert wird dann die Funktion mit dem Quellcode 4.8 aufgerufen. Sollte die Überprüfung nicht erfolgreich sein, wird es eventuell mit einem anderen zufälligen Hashwert erneut probiert. Die Anzahl dieser Wiederholungen kann man aber selbst bestimmen. Sollte danach immer noch kein passender Hashwert gefunden worden sein, wird null als bignum zurückgegeben. Man hätte auch jeden anderen Wert nehmen können.

Sobald ein passender Hashwert H gefunden wurde, kann man den Token T mit der Gleichung 2.1 berechnen. Dazu werden die Funktionen aus `galois.c` verwendet, da alle Berechnungen innerhalb von $GF(2^{128})$ geschehen müssen.

Um Fehler in Blöcken zu erkennen und eventuell auch zu korrigieren, wurde die Funktion `verify` umgesetzt. Diese Funktion ist bereits in Pseudocode auf Seite sieben in der

offiziellen MAGIC-Spezifikation [Kou+20] zu finden. Zunächst wurde ein eigener Datentyp für den Rückgabotyp dieser Funktion erstellt, da mehrere Werte zurückgegeben werden müssen. Dazu wird folgendes struct im Quellcode 4.9 verwendet.

```

1 | typedef struct {
2 |     bool error;
3 |     bool correction_successful;
4 |     bignum *ciphertext_blocks;
5 |     bignum tag;
6 | } verify_result;

```

Quellcode 4.9: Der Rückgabotyp für die verify-Funktion im MAGIC-Modus.

Dabei wird etwas von der offiziellen Spezifikation [Kou+20] abgewichen, um den Quellcode einfacher zu halten und um insbesondere die Funktionsweise auf dem Mikrocontroller sicherzustellen. Es wurde zwar probiert, es wie in der Spezifikation umzusetzen, was dann aber vor allem an der Rückgabe der leeren (null) Werten gescheitert ist. Stattdessen werden nun folgende Werte im Quellcode 4.9 zurückgegeben. Wenn kein Fehler vorliegt, wird error zu true. Des Weiteren wird correction_successful zu true, wenn kein Fehler vorliegt oder ein vorhandener Fehler korrigiert werden konnte. In ciphertext_blocks befinden sich die verschlüsselten Blöcke. Liegt kein Fehler vor, werden die Blöcke unverändert und somit fehlerhaft zurückgegeben. Wenn ein Fehler korrigiert werden konnte, findet man dort die korrigierten Blöcke. Für das Feld tag, gilt die gleiche Logik, wie bei ciphertext_blocks. In tag findet man den Token T wieder.

Zunächst wird in verify der neue Token T' mit den verschlüsselten Blöcken berechnet. In bignum.c gibt es eine Funktion, um die Gleichheit von bignum-Datentypen zu bestimmen. Diese Funktion wird genutzt, um den übergebenen Token T mit T' zu vergleichen. Wenn diese gleich sind, liegt kein Fehler vor und es wird verify_result wie oben beschrieben zurückgegeben. Ansonsten wird nun das Syndrom S durch die Gleichung 2.3 im Erweiterungskörper $GF(2^{128})$ innerhalb einer Funktion berechnet. Anschließend werden die Syndromwerte $S_1, \dots, S_n$ mithilfe der Schleife im Quellcode 4.10 bestimmt. Wenn ein Block C_i fehlerhaft ist, wird S_i zum Fehlervektor e .

```

1 | // calculate error location indicators (S_i)
2 | // S_i = error_vector -> if i == i_err
3 | for (int i = 0; i < number_of_blocks; i++) {
4 |     syndrome_values[i] = mult(syndrome, hash_key_inverse);
5 |     hash_key_inverse = mult(hash_key_inverse, original_hash_key_inverse
6 |     );
7 | }

```

Quellcode 4.10: Berechnung der einzelnen Syndromwerte.

Im weiteren Verlauf wird die Funktion locate_error aufgerufen, welche folgendermaßen im Quellcode 4.11 umgesetzt wurde. Die Variable number_of_errors wird immer inkrementiert, wenn ein Syndromwert einem Fehlervektor entspricht. Laut Kounavis u. a. [Kou+20] darf dies maximal nur einmal vorkommen, weshalb -1 zurück-

gegeben wird, wenn `number_of_errors` größer als eins ist. Dasselbe gilt, wenn alle Syndromwerte korrekt sind. Es wird der fehlerhafte Index zurückgegeben, wenn `number_of_errors` genau eins ist. Der fehlerhafte Block wird korrigiert, wenn der fehlerhafte Index (nicht -1) zurückgegeben wird. Dazu reicht eine Addition in $GF(2^{128})$ zwischen dem Fehlervektor, bzw. dem Syndromwert mit dem Index und dem fehlerhaften Block mit dem Index aus. Es wird anschließend `verify_result` mit den passenden Werten zurückgegeben, wenn der Block korrigiert werden konnte. Ansonsten wird im nächsten Schritt geschaut, ob ein Fehler im Token vorliegt.

```

1  int locate_error(bignum syndrome_values[], int number_of_blocks, int
    threshold) {
2      int number_of_errors = 0;
3      int error_index = -1;
4
5      // is only one block corrupted?
6      for (int i = 0; i < number_of_blocks; i++) {
7          if (hamming_weight(syndrome_values[i]) <= threshold) {
8              number_of_errors++;
9              error_index = i;
10         }
11         // at least two blocks are corrupted
12         if (number_of_errors > 1) {
13             return -1;
14         }
15     }
16
17     // no block is corrupted
18     if (number_of_errors == 0) {
19         return -1;
20     }
21
22     // one block is corrupted
23     return error_index;
24 }

```

Quellcode 4.11: Die Funktion, um einen fehlerhaften Block möglicherweise zu lokalisieren.

Sollte ein möglicher Fehler im Token vorliegen, werden zunächst T' und T miteinander addiert. Das Ergebnis davon entspricht dem Fehlervektor, wenn T' der korrekt berechnete Token ist und somit keine Fehler in den verschlüsselten Blöcken vorliegt. Es wird in `can_correct_parity` überprüft, ob dieses Ergebnis ein Hamming-Gewicht kleiner oder gleich dem Schwellenwert hat, da der Fehlervektor eben durch diesen gebunden ist. Warum dies genau so funktioniert, kann man bei Bedarf in der offiziellen Spezifikation [Kou+20] von MAGIC nachlesen. Es wird dann `verify_result` mit dem aktualisierten Token T' zurückgegeben. Sollte `can_correct_parity` aber feststellen, dass T nicht korrigiert werden kann, liegt ein unkorrigierbarer Fehler vor. Zum Beispiel sind dann mehrere Blöcke in $C_1, \dots, C_n$ fehlerhaft oder es liegen mehr Bit-Fehler in einem Block vor, als es durch den Schwellenwert T_{th} vorgegeben ist. Ein entsprechendes Ergebnis mit fehlerhaften Blöcken wird dann zurückgegeben.

4.6 Hamming-Code pro Block und über alle Blöcke

Der Hamming-Code wurde über alle Blöcke und pro Block umgesetzt. Es gibt nur Unterschiede in der Anzahl der Paritätsbits und in deren Positionen in den Blöcken. Das Prinzip der Fehlerkorrektur ist bei beiden Umsetzungen jedoch identisch.

Zunächst wird in einer Funktion mit dem Klartext als Input bestimmt, wie viele verschlüsselte Blöcke man braucht, um den Hamming-Code mit den zusätzlichen Paritätsbits darzustellen. Zuerst wird hier die Anzahl der Blöcke ohne Parität berechnet. Anschließend wird geschaut, wie viele zusätzliche Blöcke nötig sind, um die Blöcke mit den Paritätsbits darzustellen. Bei der Implementierung des Hamming-Codes pro Block fallen acht Paritätsbits pro Block an. Beim Hamming-Code über alle Blöcke muss man die Formel $1 + \log_2(n)$ nutzen, um die Anzahl der Paritätsbits zu bestimmen. Dabei steht n für die Gesamtanzahl der Bits ohne die Paritätsbits. Man verwendet den 2er-Logarithmus, da die Paritätsbits immer an Positionen mit 2er-Potenzen vorkommen. Bei der Implementierung pro Block muss man bei der Bestimmung der Blockanzahl außerdem darauf achten, dass man bei den zusätzlich hinzugefügten Blöcken ebenfalls die Paritätsbits beachten muss. Zum Beispiel braucht man bei vier Blöcken ohne Parität 32 zusätzliche Paritätsbits und somit einen zusätzlichen Block. Dieser Block braucht ebenfalls acht Paritätsbits, wodurch man insgesamt 40 zusätzliche Bits benötigt. Somit reicht ein zusätzlicher Block immer noch aus. Wenn man jetzt aber bei 16 Blöcken die Parität hinzufügen muss, braucht man 128 zusätzliche Bits. Also einen vollen zusätzlichen `bignum` Block. Dieser braucht acht zusätzliche Bits, wodurch man insgesamt 136 Paritätsbits benötigt. Somit braucht man insgesamt zwei zusätzliche Blöcke. Aus diesem Grund muss man den zuvor hinzugefügten Block auch mit beachten. Bei der Implementierung über alle Blöcke ist dies nicht notwendig, da die Anzahl der Paritätsbits in Bezug auf die Blockanzahl nur logarithmisch steigt. Selbst bei einer Blockanzahl von einer Million braucht man nur 27 zusätzliche Paritätsbits. Man ist also sehr weit weg, einen noch weiteren Block wie bei der Implementierung pro Block zu brauchen.

Um die Paritätsbits passend zu setzen, muss man mit der Binärdarstellung der Blöcke arbeiten. Um n `bignum`-Blöcke als Bits zu repräsentieren, wird ein 2D-Array mit dem Datentyp `bool` genutzt. Es entsteht somit ein Array mit n `bool`-Arrays, die jeweils 128 Felder haben. Dabei steht der Wert `true` für eins und `false` für null. Dadurch kann man direkt mit den Bits arbeiten, was beim Hamming-Code von Vorteil ist. Zunächst werden dazu die Bits verschoben, um Platz für die Paritätsbits an den Stellen mit Zweierpotenzen und der Stelle null zu schaffen. Dafür wird der folgende Quellcode 4.12 verwendet. Zu Beginn repräsentiert `index_full_parity` die Gesamtzahl der Bits über alle Blöcke und steht für die aktuelle Position über alle Arrays. Dagegen steht `bit_index_parity_array` für die aktuelle Bit-Position in einem bestimmten Array beginnend bei 127. Es wird mit dem letzten Block begonnen, da der Bit-Array nach vorne hin verschoben wird. Um zu überprüfen, ob die Position eine Zweierpotenz ist und

somit ausgelassen werden muss, wird die Zeile sechs genutzt. Die Position wird übersprungen und wird zu false, wenn es sich um eine Position für ein Paritätsbit handelt. Die Idee dahinter ist, dass nur das vorderste Bit bei einer Zweierpotenz gesetzt ist. Es sind dagegen alle Bits gesetzt, wenn man von der Zweierpotenz eins subtrahiert. Bei der AND-Operation auf Bitebene bleibt in diesem Fall nur die null zurück. Man kann sich damit sicher sein, dass es sich aktuell um eine Zweierpotenz handelt. Zum Beispiel würde das folgendermaßen aussehen.

$$((2^3 = 8_{10} = 1000_2) \wedge (7_{10} = 0111_2)) = 0 \quad (4.1)$$

Wenn es sich um keine Zweierpotenz handelt, wird das entsprechende Bit in einen Array von mehreren `bit_arrays` geschrieben. Sobald dieser voll ist, wird zum nächsten Bit-Array gegangen. Für die Parität pro Block wird die gleiche Logik angewendet. Dazu muss `index_full_parity` weggelassen und in Zeile sechs durch die Variable `bit_index_parity_array` ersetzt werden.

```

1 // iterate over all bit arrays starting from the back
2 for (int i = number_of_bignums-1; i >= 0; i--) {
3     // iterate over all bits
4     for (int k = 127; k >= 0; k--) {
5         // checks if bit index is used for parity and if true skip this
           position
6         if ((index_full_parity & (index_full_parity-1)) == 0) {
7             k++;
8         }
9         else {
10            bit_arrays[index_bit_arrays_parity][bit_index_parity_array]
                = bit_arrays_no_parity[i][k];
11        }
12
13        bit_index_parity_array--;
14        index_full_parity--;
15
16        // bit array with parity bits is full -> go to the next one
17        if (bit_index_parity_array == -1) {
18            index_bit_arrays_parity--;
19            bit_index_parity_array = 127;
20        }
21    }
22 }
```

Quellcode 4.12: Die Schleife, um Platz für die Paritätsbits über alle Blöcke zu schaffen.

Im nächsten Schritt werden die Werte der Paritätsbits anhand einer geraden Parität bestimmt. Zunächst wird mit allen gesetzten Bits eine XOR-Operation durchgeführt, so dass ein Resultat x entsteht, welches man schon aus Abschnitt 2.4 kennt. Anschließend werden die Bits von x genutzt, um die Paritätsbits zu setzen. Ist ein entsprechendes Bit an Position y in x eine eins, wird das Paritätsbit an Position 2^y in den verschlüsselten Blöcken zu einer eins. Nebenbei wird die Anzahl der gesetzten Bits zusammen mit den gesetzten Paritätsbits gezählt. Anhand dessen wird das Bit an Position null bestimmt. All dies kann man außerdem im Quellcode 4.13 nachvollziehen. In den Zeilen fünf bis zwölf findet die XOR-Operation statt, wobei zusätzlich mit 128 multipliziert wer-

den muss, da es sich um den Hamming-Code über alle Blöcke handelt. Des Weiteren werden in den Zeilen 16 bis 25 die Paritätsbits gesetzt, indem man sich die Bits von x anschaut. Dabei muss man das Ganze immer auf die einzelnen Blöcke reduzieren, weshalb die modulare Arithmetik und die Division mit 128 noch gemacht werden müssen. In den letzten Zeilen wird das erste Bit bestimmt, um auch zwei Bit-Fehler zu erkennen.

Für das Setzen der Paritätsbits pro Block muss das Ganze immer für jeden Block getrennt gemacht werden. Man bleibt dafür innerhalb der for-Schleife, die in Zeile fünf beginnt. Man braucht dann auch die modulare Arithmetik nicht mehr, da alles pro Block geschieht.

```

1 void set_parity_bits_all_blocks(bool bit_arrays[][128], int
  number_of_bignums_with_parity) {
2     int x = 0;
3     int number_of_set_bits = 0;
4
5     for (int i = 0; i < number_of_bignums_with_parity; i++) {
6         for (int k = 0; k < 128; k++) {
7             if (bit_arrays[i][k]) {
8                 x ^= (k + (128 * i));
9                 number_of_set_bits++;
10            }
11        }
12    }
13
14    int number_of_parity_bits = (int)log2(number_of_bignums_with_parity
    * 128);
15
16    for (int k = 0; k < number_of_parity_bits; k++) {
17        if ((x >> k) & 1) {
18            int index_parity_bit = (int)pow(2, k);
19            bit_arrays[index_parity_bit / 128][index_parity_bit % 128]
            = !(bit_arrays[index_parity_bit / 128][index_parity_bit
            % 128]);
20
21            if (bit_arrays[index_parity_bit / 128][index_parity_bit %
            128]) {
22                number_of_set_bits++;
23            }
24        }
25    }
26
27    if (number_of_set_bits % 2 == 1) {
28        bit_arrays[0][0] = true;
29    }
30 }

```

Quellcode 4.13: Das Setzen der Paritätsbits über alle Blöcke.

Ob die Anzahl der gesetzten Bits nun gerade oder ungerade ist, kann man durch die XOR-Operation bestimmen, da XOR eine Addition mod 2 ist. Somit kann ermittelt werden, ob die Anzahl der Bits in x an Position y gerade oder ungerade ist. Da die Paritätsbits an Zweierpotenzen stehen und alle deren zugehörigen Bits an Positionen stehen, deren Binärdarstellung dort ebenfalls ein gesetztes Bit haben, kann man mit der XOR-Operation ermitteln, ob die Anzahl der Bits dieser bestimmten Paritätsgruppe nun gerade oder ungerade ist. Wenn alle gesetzten Bits der Nachricht mit XOR zusammengefasst werden, spiegelt x an Position y genau eine Paritätsgruppe wider, da diese Stelle nur

von Bits dieser Gruppe beeinflusst werden kann. Wenn an Position y in der Binärdarstellung von x eine eins steht und somit die Parität ungerade ist, muss das Paritätsbit an der Position 2^y gesetzt werden. Bei erneuter Ausführung der XOR-Operation im Quellcode 4.13 in der ersten for-Schleife wird bei x in der Binärdarstellung an Position y nun eine null stehen, da die Parität jetzt in dieser Gruppe gerade ist [3Bl20].

Im nächsten Schritt werden die Arrays aus `bit_arrays` aus dem Quellcode 4.13 wieder in einen `bignum`-Array umgewandelt. Dazu wird eine Funktion aufgerufen, die unter anderem durch Left-Shifts um eins die `bignum`-Datentypen erstellt. Durch die Left-Shifts werden die Bit, bzw. `bool`-Werte an die richtige Position verschoben. Alle diese genannten Konvertierungen und das Setzen der Paritätsbits werden in einer Funktion `add_parity_...` bewerkstelligt, indem die eben genannten Funktionen aus diesem Abschnitt dort aufgerufen werden. Man kann die beschriebenen Funktionen als Hilfsfunktionen betrachten. Es gibt jeweils eine `add_parity_...` Funktion über alle Blöcke und eine pro Block. Letztendlich muss man diese Funktion aufrufen und es werden passende Paritätsbits zu einem `bignum`-Array hinzugefügt. Man muss somit nicht die Bit-Arrays direkt nutzen. Die Implementierung mit den Bit-Arrays hat jedoch den Nachteil, dass noch mehr Speicherplatz benötigt wird.

Nachdem man die Paritätsbits gesetzt hat, kann man überprüfen ob Bitfehler vorliegen und ob diese korrigiert werden können. Dazu wird die Funktion `verify` verwendet. Ebenfalls wird wie im MAGIC-Modus ein Token zu Beginn überprüft, um die Authentizität sicherstellen zu können. Dadurch kann direkt ausgeschlossen werden, ob überhaupt Bitfehler vorliegen. Es wird dasselbe Verfahren zur Tokengenerierung wie bei MAGIC genutzt, um nur wirklich die Fehlerkorrektur der beiden Verfahren miteinander zu vergleichen. Für die Tokengenerierung reicht beim Hamming-Code ein beliebiger Hashwert aus. Es gibt jeweils wieder eine Funktion für den Hamming-Code über alle Blöcke und eine pro Block. Ähnlich zum MAGIC-Modus aus Abschnitt 4.5 wird ein selbst erstellter Rückgabetypp in Form eines `struct` verwendet, um alle möglichen Ergebniszustände abdecken zu können. Diesen kann man im folgenden Quellcode 4.14 sehen. Die Variable `correction_successful` wird zu `true`, wenn kein Fehler vorliegt oder ein 1-Bit-Fehler korrigiert werden kann. Da ein 1-Bit-Fehler durch den Hamming-Code immer korrigiert werden kann, ist es eigentlich nicht unbedingt notwendig, die genannte Variable noch zusätzlich neben `one_bit_error` auf `true` zu setzen. Es wird trotzdem gemacht, da man dadurch feststellen kann, ob überhaupt ein Fehler vorliegt. Die Variable `two_bit_error` wird zu `true`, wenn ein 2-Bit-Fehler erkannt wird. Sollten jedoch mehr als 2-Bit-Fehler vorliegen, funktioniert die Funktion `verify` nicht mehr korrekt. Zum Beispiel wird ein 3-Bit-Fehler als 1-Bit-Fehler interpretiert. Dies liegt jedoch nicht an der Funktion `verify`, sondern am Hamming-Code an sich.

```
1 | typedef struct {  
2 |     bool correction_successful;  
3 |     bool one_bit_error;  
4 |     bool two_bit_error;
```

```
5 |     bignum *ciphertext_blocks_with_parity;  
6 | } hc_result;
```

Quellcode 4.14: Der Rückgabetypp zum Verifizieren des Hamming-Codes.

Zu Beginn der Funktion `verify` wird der Token überprüft, indem der schon bekannte Token T der Funktion übergeben wird. Daraufhin wird ein neuer Token T' anhand der weiteren Werte berechnet. Sollte T und T' identisch sein, kann man Bitfehler jeglicher Art direkt ausschließen. Des Weiteren wird die Authentizität dadurch sichergestellt. In dieser Funktion wird ebenfalls mit den Bit-Arrays gearbeitet, indem die verschlüsselten Blöcke umgewandelt werden. Um mögliche Fehler zu erkennen und zu korrigieren, wird die gleiche Logik wie zum Setzen der Paritätsbits verwendet. Zunächst werden alle gesetzten Bits mit der XOR-Operation zusammengefasst und deren Anzahl wie im Quellcode 4.13 gezählt. Wenn das Ergebnis x null ist, liegt kein Fehler vor. Eine Paritätsgruppe spiegelt ein Bit in x wider. Sind alle Paritätsgruppen somit gerade, wird das Resultat x durch die XOR-Operation nur Nullen anzeigen. Ob ein Fehler vorliegt, wurde bereits durch die Tokenüberprüfung sichergestellt. Sollte x nicht null und die Anzahl der gesetzten Bits ungerade sein, liegt ein 1-Bit-Fehler vor. Das Ergebnis x spiegelt die Position des fehlerhaften Bits wider. Es findet im Prinzip eine XOR-Operation zwischen der null und dem fehlerhaften Bit statt. Das Ergebnis ist die Position des 1-Bit-Fehlers. Dieser wird korrigiert, indem eine Negation des `bool`-Wertes an der herausgefundenen Position stattfindet. Diese Blöcke werden anschließend korrigiert zurückgegeben. Ist die Anzahl der gesetzten Bits gerade und x nicht null, liegt ein 2-Bit-Fehler vor. Durch den ersten Fehler wird die Anzahl der gesetzten Bits ungerade und durch den zweiten Fehler wird diese wieder gerade. Die fehlerhaften Blöcke werden in diesem Fall unverändert zurückgegeben.

Zu guter Letzt gibt es jeweils eine Funktion, um die Paritätsbits der verschlüsselten Blöcke wieder zu entfernen, sodass man damit den ursprünglichen Text erneut erzeugen kann. Wie man anschließend zum Klartext kommt, kann man in Abschnitt 4.4 nachlesen.

4.7 Arbeiten mit dem Mikrocontroller

Für diese Arbeit wird ein Cortex-M3-Prozessor mit externen SRAM genutzt. Genauer gesagt das CW521 Ballistic Gel Board, welches im Folgenden als Mikrocontroller bezeichnet wird. Für die richtige Funktionsweise des Mikrocontrollers wird ein externes Repository [Inc22] genutzt. Dadurch muss man nur die Funktionsaufrufe für den MAGIC-Modus und den Hamming-Code in der Datei `main_cw521.c` durchführen. Wie man den Mikrocontroller startet, kann man in Kapitel 3 nachlesen. Der Mikrocontroller wird genutzt, um die Evaluierungen durchzuführen. Dazu wurden am Anfang der Datei mehrere Funktionen definiert. Einerseits, um die Takte auf dem Mikrocontroller abzurufen. Andererseits, um die Operationen mit verschiedenen Parametern zu evaluieren.

Genaueres zur Evaluierung findet man in Kapitel 5. Außerdem ist es mit dem Mikrocontroller nicht möglich, Output in der Konsole zu erzeugen. Dazu bräuchte man zusätzliche Hardware. Anstatt dessen wird gdb für die Konsolenausgabe genutzt. Dessen Output wird durch den Computer erzeugt, an den der Mikrocontroller angeschlossen ist.

Innerhalb der main-Funktion der Datei `main_cw521.c` werden die Funktionen aufgerufen, sodass diese auf dem Mikrocontroller ausgeführt werden. Die Funktionsaufrufe sind am Ende zwischen `ballistic_register_handlers` und der `while`-Schleife zu finden. Dort können auch die Tests ausgeführt werden. Wie die Tests genau funktionieren, folgt im nächsten Abschnitt 4.8.

Um die Dateien aus Abschnitt 4.1 in `main_cw521.c` nutzen zu können, musste das Makefile des Mikrocontrollers folgendermaßen angepasst werden.

```

1 | MAGIC = ./magic_bachelorarbeit/src
2 | MAGIC_FILES = $(MAGIC)/galois.c $(MAGIC)/bignum.c $(MAGIC)/
   |   util_functions.c $(MAGIC)/salsa20.c $(MAGIC)/magic_mode.c $(MAGIC)/
   |   hamming_code.c $(MAGIC)/examples.c
3 |
4 | MAGIC_TEST = ./magic_bachelorarbeit/test
5 | MAGIC_TEST_FILES = $(MAGIC_TEST)/test_bignum.c $(MAGIC_TEST)/
   |   test_galois.c $(MAGIC_TEST)/test_hamming_code.c $(MAGIC_TEST)/
   |   test_magic_mode.c $(MAGIC_TEST)/test_salsa20.c $(MAGIC_TEST)/
   |   test_util_functions.c
6 |
7 | SRC += main_cw521.c naeusb_ballistic.c naeusb/naeusb.c naeusb/
   |   naeusb_default.c $(MAGIC_FILES) $(MAGIC_TEST_FILES)

```

Quellcode 4.15: Angepasste Zeilen im Makefile des Mikrocontrollers.

Dazu muss man zunächst im Quellcode 4.15 das passende Verzeichnis angeben. Anschließend werden die Dateien, die in `main_cw521.c` genutzt werden, dem SRC-Target hinzugefügt. Außerdem werden diese noch folgendermaßen in `main_cw521.c` inkludiert.

```
#include "magic_bachelorarbeit/src/[FILE].h"
```

Dazu wird `[FILE]` durch die jeweilige Datei ersetzt. Danach muss man `make` aufrufen, um die Funktionen der Dateien aus 4.1 zu nutzen.

4.8 Die Funktionsweise der Tests

Es wurden Unittests für die einzelnen Funktionen erstellt. Pro Datei aus 4.1 gibt es eine Datei mit Unittests. Mit den Tests soll überprüft werden können, ob alle Programmteile korrekt funktionieren, nachdem Änderungen am Quellcode vorgenommen wurden.

Ein Großteil der Tests arbeitet nach dem Prinzip Arrange-Act-Assert. In **Arrange** werden alle benötigten Inputparameter für die zu testende Funktion erstellt. Innerhalb

von **Act** wird die zu testende Funktion ausgeführt. Zu guter Letzt werden in **Assert** die Ergebnisse der Funktion auf ihre Richtigkeit überprüft. Es wurde sich für dieses Prinzip entschieden, da man damit gut strukturierte und funktional gute Tests erstellen kann. Um die Ergebnisse zu prüfen, wird `assert` aus der C-Standardbibliothek verwendet. Jeder Test ist durch eine eigene Funktion repräsentiert. Pro Testdatei gibt es zudem eine Funktion `x` am Ende, die alle Testfunktionen der entsprechenden Datei aufruft. Diese Funktion wird wiederum dafür genutzt, um alle diese Tests in der Datei `test_main.c` in einer `main`-Funktion auszuführen. Diese Funktionen `x` werden auch dazu genutzt, um die Tests auf dem Mikrocontroller auszuführen, indem diese Funktionen in `main_cw521.c` aufgerufen werden. Demzufolge wird auch kein externes Test-Framework verwendet. Es müsste ebenfalls auf dem Mikrocontroller laufen und es wäre im Verlauf dieser Arbeit zu zeitaufwendig, dies zu realisieren.

Es wurden für `bignum` alle Funktionen bis auf die `print`-Funktionen getestet. Es wurde außerdem darauf geachtet, dass die Funktionen auch beim Überschreiben einer Variable wie erwartet funktionieren. Dies sieht man beispielhaft am folgenden Test im Quellcode 4.16. Der Grund dessen ist, dass dies in der Entwicklung nicht immer richtig funktioniert hat. Die Lösung war es, den Eingangsparameter wie schon in Abschnitt 4.2 zuvor zu kopieren. Diese korrekte Funktionalität wird auch für die Funktionen in `galois.c` getestet.

```

1 void test_shift_right_by_one_overwrite_variable() {
2     uint32_t ahex[] = {0x54dbcf8f, 0xfd761f7c, 0x2c239ae3, 0x2732819c};
3     uint32_t expected[] = {0x2a6de7c7, 0xfebb0fbe, 0x1611cd71, 0
4         x939940ce};
5
6     bignum a = init_bignum(ahex);
7
8     a = shift_right_by_one_bignum(a);
9
10    for (int i = 0; i < 4; i++) {
11        assert(a.chunks[i] == expected[i]);
12    }
13 }
```

Quellcode 4.16: Ein beispielhafter Test.

Aus `galois.c` werden alle Funktionen bis auf die `power`-Funktion getestet. Der Grund ist, dass `power` nur eine Hilfsfunktion für `mult_inverse` darstellt. Die Funktion zum Finden des inversen Elementes wird ganz normal getestet, wodurch `power` ebenfalls validiert wird. Um die restlichen Funktionen aus `galois.c` zu testen, wurden mit Sage¹ zufällige Werte aus dem Erweiterungskörper $GF(2^{128})$ generiert. Im folgenden Quellcode 4.17 sieht man, wie diese Zahlen mit Sage für die Tests erstellt werden. So war es möglich, die Funktionen aus `galois.c` mit repräsentativen Werten zu testen.

```

1 sage: K.<a> = GF(2^128, modulus=x^128 + x^7 + x^2 + x + 1)
2 sage: x = K.random_element()
3 sage: x
```

¹Sage ist eine Mathesoftware, welche in der Konsole ausgeführt werden kann.

```

4 | a^127 + a^123 + a^122 + a^120 + a^119 + a^118 + a^117 + a^115 + a^113 +
   | a^112 + a^111 + a^110 + a^107 + a^104 + a^103 + a^102 + a^100 + a
   | ^99 + a^98 + a^96 + a^95 + a^94 + a^93 + a^92 + a^89 + a^83 + a^81 +
   | a^79 + a^76 + a^75 + a^73 + a^72 + a^69 + a^68 + a^67 + a^66 + a^65
   | + a^63 + a^60 + a^57 + a^55 + a^54 + a^50 + a^49 + a^48 + a^47 + a
   | ^46 + a^44 + a^43 + a^37 + a^36 + a^34 + a^33 + a^24 + a^22 + a^21 +
   | a^19 + a^9 + a^7 + a^5 + a^4 + a^2 + 1
5 | sage: x.integer_representation()
6 | 188645431516091266966356573972127548085

```

Quellcode 4.17: Die Generierung von Testwerten zum Testen der Arithmetik in endlichen Erweiterungskörpern.

In `magic_mode.c` wird nicht jede einzelne Funktion für sich getestet. Viele Funktionen werden innerhalb der Datei über andere Funktionen aufgerufen. Es wird die Ver- und Entschlüsselung des Tokens getestet, da die Tokengenerierung für MAGIC und für den Hamming-Code wichtig ist. Insbesondere war es sehr trivial, diese Funktion zu testen. Zum anderen wird die `verify`-Funktion mit vielen verschiedenen Eingangsparametern getestet. Unter anderem die Korrektur von einem 1-Bit-Fehler und die richtige Funktionsweise, wenn kein Fehler vorliegt. Die nicht getesteten Funktionen werden vor allem in `verify` genutzt. Diese werden durch die Tests der `verify`-Funktion ebenfalls validiert. Bei keinem dieser Tests wird ein zufälliger Hashwert bestimmt und überprüft. Man könnte sonst nie sicher sein, ob ein passender Hashwert gefunden wird. Sollte keiner gefunden werden, schlägt der Test nur aufgrund dessen fehl. Dies soll aber nicht der Fall sein. Außerdem wäre der Test nicht mehr deterministisch, da dieser dann immer mit anderen Werten läuft und ohne Änderungen am Quellcode fehlschlagen kann.

In den Tests der `util`-Funktionen wird vor allem das Finden eines passenden Hashwertes durch die Funktion `check_hash_key` getestet. Um einen nicht passenden Hashwert zu validieren, wird der Hashwert gleich null gesetzt. Anhand der Gleichung 2.2 sieht man, dass dieser für MAGIC nicht funktionieren wird. Für den gegenteiligen Fall wird ein Hashwert verwendet, von dem man weiß, dass dieser für MAGIC passend ist. Ansonsten werden die restlichen `util`-Funktionen ohne weitere Besonderheiten getestet.

Zum Testen der Salsa20-Implementierung werden Testvektoren aus der offiziellen Spezifikation [Ber05] und KATs [Boc12] verwendet. Des Weiteren wird in diesem Kontext auch die Verschlüsselung von Klartext zu verschlüsselten Blöcken und anschließend deren Entschlüsselung getestet. Es wird in diesem Test überprüft, ob die entschlüsselten Blöcke gleich dem ursprünglichen Klartext sind.

Beim Testen des Hamming-Codes wird jeweils die Version pro Block und die Version über alle Blöcke im gleichen Ausmaß überprüft. Alle Funktionen wurden hier einzeln getestet, da diese zum Teil ziemlich komplex sind. Zum Teil wurden für die Tests Bit-Arrays angelegt. Bei der normalen Benutzung ist dies nicht notwendig. Ansonsten gibt es hier keine Besonderheiten.

Wenn man die Tests durch die von `make test` erstellte Datei auf dem normalen Computer ausführt, sollte man keinen Output sehen. Ist das der Fall, sind alle Tests erfolgreich gewesen. Wenn ein bestimmter Test fehlschlägt, bekommt man eine Fehlermeldung

durch das `assert`-Statement. In der Konsole sieht man die Datei und die Zeile des fehlschlagenden Tests. Wenn man die Tests auf dem Mikrocontroller ausführt, sieht man keinen Output. Nach deren Ausführung in der Datei `main_cw521.c` wird in eine Endlosschleife gegangen. Anschließend muss man das Programm aktiv mit `CTRL+C` unterbrechen. Wenn alle Tests erfolgreich durchgelaufen sind, sollte man sehen, dass sich das Programm in dieser Endlosschleife befunden hat. Sollte vorher durch einen fehlerhaften Test `assert` nicht erfolgreich gewesen sein, geht das Programm in einen standardmäßigen Interrupt-Handler. Dies kann man ebenfalls durch die Unterbrechung des Programms erkennen. Jedoch fehlt die Information, welcher Test fehlgeschlagen ist.

Kapitel 5

Evaluierung

In diesem Kapitel wird zu Beginn vorgestellt, wie die Takte auf dem Mikrocontroller ermittelt werden. Anschließend wird gezeigt, wie die Takte konkret für die jeweiligen Operationen des MAGIC-Modus und des Hamming-Codes gemessen werden. Dazu wurden die Anzahl der Blöcke und die Anzahl der zu korrigierbaren Bits variiert. Zum Schluss werden die jeweiligen Speicherkosten der beiden Verfahren aufgezeigt.

5.1 Die Bestimmung der Takte

Um die Rechenleistung des MAGIC-Modus und des Hamming-Codes zu evaluieren, werden die Takte für die einzelnen Operationen gemessen. Dazu werden die Register DWT_CTRL, DWT_CYCCNT und DEMCR des Cortex-M3-Prozessors genutzt. Hier steht DWT für die Debug-Einheit des Prozessors. Mit dem Register DWT_CTRL wird diese Debug-Einheit aktiviert und mit dem Register DWT_CYCCNT werden die Takte abgerufen. Das DEMCR wird genutzt, um die Nutzung aller DWT Register zu aktivieren. Im Quellcode 5.1 sieht man die Adressen der jeweiligen Register, die immer 32-Bit groß sind. In Zeile fünf werden die DWT Register aktiviert und in den Zeilen sechs bis sieben werden die Werte in den Registern zurückgesetzt. Diese Funktion aus dem Quellcode 5.1 wird immer vor dem Starten des Messens aufgerufen, um saubere Werte zu bekommen [Devb] [Deva].

```
1 | void reset_timer() {  
2 |     DWT_CYCCNT = (volatile uint32_t *)0xE0001004;  
3 |     DWT_CTRL = (volatile uint32_t *)0xE0001000;  
4 |     DEMCR = (volatile uint32_t *)0xE000EDFC;  
5 |     *DEMCR = *DEMCR | 0x01000000;  
6 |     *DWT_CTRL = 0;  
7 |     *DWT_CYCCNT = 0;  
8 | }
```

Quellcode 5.1: Die Funktion, um die Register zu konfigurieren.

Um das Hochzählen der Takte zu starten, wird das rechte Bit vom DWT_CTRL Register zu

einer eins. Das Setzen von diesem Bit hat die Wirkung, dass das Register DWT_CYCCNT aktiviert wird und nun die Takte hochgezählt werden. Dies geschieht immer direkt nach dem Aufruf von `reset_timer` aus dem Quellcode 5.1. Um das Hochzählen der Takte wieder zu stoppen, wird das rechte Bit vom DWT_CTRL Register zu einer null. Dadurch wird das DWT_CYCCNT Register deaktiviert. Das Ganze geschieht immer nach der Operation, von der die Takte bestimmt werden sollen. Um im Anschluss die Anzahl der Takte abzurufen, wird der aktuelle Wert aus dem DWT_CYCCNT Register ausgelesen [Devb] [Deva].

5.2 Vorgehensweise zur Bestimmung der Takte für eine Operation

Die gemessenen Operationen sind in einzelne Funktionen aufgeteilt, die am Anfang der Datei `main_cw521.c` zu finden sind. Mithilfe einer `for`-Schleife wird die Anzahl der Blöcke, die Anzahl der Bitfehler, bzw. der Schwellenwert oder fehlerhafte Blockindex variiert. Das Intervall, den Start- und Endpunkt dieser Messparameter kann man über die Funktionsparameter festlegen. Des Weiteren kann man die Anzahl der Messungen pro Blockanzahl oder pro Anzahl der Bitfehler der Funktion übergeben. Durch mehrere Messungen fallen Messfehler und Ausreißer weniger stark ins Gewicht. Davon werden zusätzlich der Durchschnitt und die Standardabweichung ermittelt, um eine Aussage über die Güte der Messwerte treffen zu können. Es wird dabei immer die Anzahl der Takte wie in Abschnitt 5.1 gemessen. Im Quellcode 5.2 werden die Takte beispielhaft für die Verschlüsselung bei einer variierenden Blockanzahl ermittelt. Zunächst wird über die Blockanzahl iteriert. Für eine bestimmte Blockanzahl werden mehrmals die Takte ab Zeile sieben gemessen. Die eigentliche Operation in Zeile zwölf ist von Funktionen zur Ermittlung der Takte umklammert. In Zeile 17 wird die Taktanzahl in einem Array gespeichert, um den Durchschnitt in Zeile 20 und die Standardabweichung in Zeile 21 zu bestimmen. Es wird an einigen Stellen `volatile` genutzt, um mit `gdb` die Werte ausgeben zu lassen. Ansonsten werden die Werte in Zeile fünf und 14 wegoptimiert.

```

1 | void encryption_changing_blocks(int start, int end, int interval, int
   |     number_of_measurements) {
2 |     [...]
3 |     // iterate over blocks
4 |     for (int i = start; i <= end; i += interval) {
5 |         volatile uint32_t results[number_of_measurements];
6 |         // measure the cycle count [number_of_measurements] times
7 |         for (int f = 0; f < number_of_measurements; f++) {
8 |             [...]
9 |             reset_timer();
10 |            start_timer();
11 |
12 |            plaintext_to_ciphertext_blocks(blocks, text, key, nonce);
13 |
14 |            volatile uint32_t number_of_cycles = get_cycles();

```

```

15     stop_timer();
16
17     results[f] = number_of_cycles;
18 }
19
20 volatile float avg = average(results, number_of_measurements);
21 volatile float sd = standard_deviation(results,
22     number_of_measurements, avg);
23 [...]
24 }

```

Quellcode 5.2: Eine repräsentative Ermittlung der Takte für eine Operation.

Alle weiteren Funktionen zur Ermittlung der Takte sind wie im Quellcode 5.2 aufgebaut. Sie unterscheiden sich nur darin, dass die Operation eine andere ist oder über andere Werte als die Blockanzahl iteriert wird. Außerdem können sie sich in den Parametern und den notwendigen Code vor der eigentlichen Operation unterscheiden. Vor der eigentlichen Operation werden vor allem die `bignum`-Blöcke oder Text mit der richtigen Länge erstellt. Dazu gibt es häufig einen `bignum`-Block, der noch vor den `for`-Schleifen in Zeile zwei im Quellcode 5.2 initialisiert wird. In der Zeile acht wird dieser `bignum`-Block mehrmals einem `bignum`-Array hinzugefügt. Bei einem String verhält sich dies sehr ähnlich. Um die Funktionen auszuführen, werden sie zwischen `ballistic_register_handlers()` und der `while`-Schleife in der `main`-Funktion der Datei `main_cw521.c` aufgerufen.

Es werden Parameter anstatt feste Werte verwendet, um die Operationen schnell mit verschiedenen Werten evaluieren zu können. Zum Beispiel kann man so die Anzahl der Messungen anpassen, ohne die eigentliche Funktion zu verändern. Dies ist vor allem notwendig, wenn die Operation länger als erwartet läuft. Man möchte dann aus Zeitgründen weniger Messungen durchführen, was man so sehr einfach machen kann.

5.3 Performance des MAGIC-Modus

Für die Bestimmung der Performance des MAGIC-Modus werden die Anzahl der Takte pro Operation gemessen. Zu den Operationen gehört das Finden eines Hashwertes, die Tokengenerierung und das Verifizieren der verschlüsselten Blöcke. Dazu werden Funktionen wie im Abschnitt 5.2 genutzt. Es wird eine Blockanzahl von eins bis zehn in den Messungen verwendet, da dies für eine Aussage der Messergebnisse bereits ausreicht. Würde man höhere Werte verwenden, dauert es einfach nur länger ohne weiteren Mehrwert für die Evaluierung zu bringen.

In Tabelle 5.1 sieht man die Anzahl der Takte, um einen passenden Hashwert zu finden. Zum einen wird die Blockanzahl und zum anderen wird der Schwellenwert variiert. Die Funktion für die Ermittlung eines Hashwertes wird immer nur mit einem Versuch aufgerufen. In der Funktion zum Messen der Takte wird außerdem überprüft, ob ein

passender Hashwert gefunden wurde. Alle Werte in der Tabelle 5.1 basieren darauf, dass ein passender Hashwert gefunden wurde. Ein nicht korrekter Hashwert fließt somit nicht in die Messergebnisse ein. Bei einer steigenden Blockanzahl steigt die Anzahl der Takte für diese Operation ebenfalls an. Wie man außerdem sehen kann, ist die Taktanzahl sehr hoch. Wenn der Schwellenwert ansteigt, muss man sehr lange warten, um einen passenden Hashwert zu bekommen. Bei einem Schwellenwert von drei dauert die Operation zum Beispiel mindestens eine bis zwei Stunden. Selbst, wenn es sich um nur einen Block handelt. Ab einem Schwellenwert von vier war es nicht mehr praktikabel, einen passenden Hashwert zu finden. Die Messung wurde bei diesem Zeitpunkt abgebrochen. Es kann auch sein, dass bei einem Schwellenwert von drei ein Overflow des Registers DWT_CYCCNT geschehen ist. Bei einem solchen Szenario werden die Takte wieder von null aus hochgezählt.

Finden eines Hashwertes			
Schwellenwert = 1 Messungen = 30		Blockanzahl = 1 Messungen = 2	
Blockanzahl	Takte (Mio.)	Schwellenwert	Takte (Mio.)
1	123,1	1	126,6
2	192,3	2	2724,0
3	261,4	3	2966,4
4	330,6	4	-
5	399,7	5	-
6	468,9	6	-
7	538,0	7	-
8	607,1	8	-
9	676,1	9	-
10	745,4	10	-

Tabelle 5.1: Anzahl der Takte in Millionen zum Finden eines Hashwertes.

Um einen Token aus den verschlüsselten Blöcken und einem gegebenen Hashwert zu generieren, braucht es viel weniger Takte als das Finden eines passenden Hashwertes. Dies kann man anhand der Tabelle 5.2 sehen. Bei einer steigenden Blockanzahl steigt die Anzahl der Takte linear an. Um die Funktion aufzurufen, braucht es ebenfalls einen Hashwert. Dazu wird ein beliebiger Hashwert ohne weitere Überprüfung verwendet. Damit wird die lange Bestimmung eines passenden Hashwertes umgangen. Außerdem kann man dadurch diese Messwerte auch für den Hamming-Code verwenden, da dort immer ein beliebiger Hashwert verwendet wird, um den Token zu generieren. Ebenfalls wird ein beliebiger Wert für die autorisierten Daten für die Messung benutzt. Eine Variation des Schwellenwertes findet hier nicht statt, da dieser zur Bestimmung eines Tokens nicht erforderlich ist.

Tokengenerierung			
Messungen = 50		Messungen = 30	
Blockanzahl	Takte (Mio.)	Blockanzahl	Takte (Mio.)
1	0,3	10	3,0
2	0,6	15	4,5
3	0,9	20	6,1
4	1,2	25	7,6
5	1,5	30	9,1
6	1,8	35	10,6
7	2,1	40	12,1
8	2,4	45	13,6
9	2,7	50	15,1

Tabelle 5.2: Tokengenerierung für unterschiedlich große Blockzahlen.

In der horizontalen Tabelle 5.3 sieht man die Anzahl der Takte, um den MAGIC-Modus zu verifizieren. Dazu werden unterschiedliche Fehlerarten genutzt. Die Anzahl der Blöcke, der Index des fehlerhaften Blocks und der Schwellenwert ändern sich hierbei. Bei allen Fehlerarten für eine steigende Blockanzahl steigt die Taktanzahl linear an. Zunächst ändert sich die Anzahl der Blöcke in den ersten vier Messreihen. Wenn kein Fehler vorliegt, ist die Verifizierung ziemlich schnell fertig, da man die Syndromwerte nicht weiter bestimmen muss. Ansonsten dauert es bei den anderen Fehlerarten ungefähr gleich lang. Diese starten bei ca. 40 Millionen Takte für einen Block und enden bei ca. 50 Millionen Takte bei zehn Blöcken. Wenn ein Fehler im Token vorliegt, ist `verify` etwas schneller fertig. Leider wurde kein Grund gefunden, warum dies der Fall sein könnte. In der Praxis macht dies jedoch keinen nennenswerten Unterschied. Der unkorrigierbare Fehler wurde so umgesetzt, dass ein Fehler im ersten und zweiten Block vorliegt. Bei den nächsten zwei Messreihen wird der fehlerhafte Block variiert. Wie man in Tabelle 5.3 sieht, spielt es keine Rolle, in welchem Block der Fehler vorliegt. In den letzten drei Messreihen wird der Schwellenwert, bzw. die Anzahl der Bitfehler variiert. Der Hashwert wurde so gewählt, sodass dieser für diese Messreihen anhand von Gleichung 2.2 passend ist. Hier spielt es ebenfalls keine Rolle, wie viele Bitfehler vorliegen. Es dauert immer gleich lang.

Zu guter Letzt befinden sich in Tabelle 5.4 die Takte, um den MAGIC-Modus komplett durchlaufen zu lassen. Dazu wird ein 1-Bit-Fehler verwendet, der korrigiert werden soll. Für eine steigende Blockanzahl steigt die Anzahl der Takte linear an. Zunächst werden dafür zufällige Schlüssel- und Noncewerte generiert. Anschließend wird der Klartext verschlüsselt und danach versucht, einen passenden Hashwert zu finden. Ist das geschehen, wird der Token generiert. Im Anschluss wird ein Bit modifiziert und danach werden die verschlüsselten Blöcke durch die `verify`-Funktion wieder korrigiert. Da bei einem solchen Durchlauf auch der Hashwert überprüft wird, wurde auf

MAGIC verifizieren												
error in ciphertext	Schwellenwert = 1 Messungen = 50	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	40,6	41,6	42,7	43,8	44,8	45,9	47,0	48,0	49,1	50,1
error in token/tag	Schwellenwert = 1 Messungen = 50	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	40,5	41,3	42,2	43,1	44,0	44,9	45,8	46,7	47,5	48,4
no error	Schwellenwert = 1 Messungen = 50	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,4	0,7	1,0	1,4	1,7	2,1	2,4	2,7	3,1	3,4
uncorrectable error	Schwellenwert = 1 Messungen = 10	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	40,6	41,7	42,7	43,8	44,8	45,8	46,9	48,0	49,0	50,0
correctable error	Schwellenwert = 1 Blockanzahl = 10 Messungen = 30	Index fehlerhafter Block	0	1	2	3	4	5	6	7	8	9
		Takte (Mio.)	50,0	49,8	49,9	50,1	49,9	49,9	49,9	49,9	49,9	50,0
error in two blocks	Schwellenwert = 1 Blockanzahl = 10 Messungen = 30	Index 2. fehlerhafter Block	0	1	2	3	4	5	6	7	8	9
		Takte (Mio.)	49,1	50,1	50,1	50,1	50,2	50,2	50,1	50,2	50,2	50,1
error in ciphertext	Blockanzahl = 4 Messungen = 30	Schwellenwert (Anzahl Bitfehler)	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	43,1	42,4	43,1	42,5	43,1	42,5	43,1	42,5	43,1	42,5
error in token/tag	Blockanzahl = 4 Messungen = 30	Schwellenwert (Anzahl Bitfehler)	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	43,4	43,4	43,4	43,4	43,4	43,4	43,4	43,4	43,4	43,4
uncorrectable error	Blockanzahl = 4 Messungen = 30	Schwellenwert (Anzahl Bitfehler pro Block)	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	44,0	44,1	44,0	43,4	44,0	44,0	44,0	43,4	44,0	44,0

Tabelle 5.3: Die Takte, um MAGIC mit unterschiedlichen Fehlerarten und Variationen zu verifizieren.

eine Variation des Schwellenwertes verzichtet. Das Finden eines passenden Hashwertes würde bereits ab einem Schwellenwert von zwei zu lange dauern, was man anhand von Tabelle 5.1 sehen kann.

MAGIC kompletter Durchlauf	
Schwellenwert = 1	
Messungen = 10	
Blockanzahl	Takte (Mio.)
1	168,3
2	241,4
3	314,7
4	387,8
5	461,0
6	534,1
7	607,4
8	680,5
9	753,8
10	826,9

Tabelle 5.4: Ein kompletter Durchlauf des MAGIC-Modus mit einem 1-Bit-Fehler.

5.4 Performance des Hamming-Codes

Für die Bestimmung der Performance des Hamming-Codes wird die Anzahl der Takte pro Operation gemessen. Zu den Operationen gehört das Hinzufügen der Paritätsbits, das Entfernen der Paritätsbits und die Fehlerkorrektur. Es wird jeweils der Hamming-Code pro Block und über alle Blöcke betrachtet. Dazu werden Funktionen wie im Abschnitt 5.2 genutzt. Im Allgemeinen lässt sich jetzt schon sagen, dass der MAGIC-Modus mehr Takte als der Hamming-Code bei der Fehlerkorrektur benötigt.

In Tabelle 5.5 sieht man die Anzahl der Takte für eine steigende Anzahl von Blöcken. Einmal für den Hamming-Code pro Block und einmal für den Hamming-Code über alle Blöcke. Grundsätzlich steigen die Werte bei einer steigenden Blockanzahl linear an. Jedoch gibt es Schwankungen für den Faktor der linearen Steigung. Dafür gibt es aktuell noch keine Erklärung. Dabei fällt außerdem auf, dass es schneller ist, Paritätsbits über alle Blöcke hinzuzufügen. Das liegt vor allem daran, dass weniger Paritätsbits benötigt werden. Außerdem braucht der Hamming-Code pro Block alle 16 Blöcke immer einen weiteren Block, wodurch über mehr Blöcke iteriert werden muss. Dies wurde auch bereits in Abschnitt 4.6 genauer betrachtet. Deswegen wird eine Blockanzahl von zehn bis 100 Blöcken in der Tabelle 5.6 verwendet, sodass man dieses Phänomen sehen kann.

Paritätsbits hinzufügen			
Messungen = 30			
pro Block		über alle Blöcke	
Blockanzahl	Takte (Tsd.)	Blockanzahl	Takte (Tsd.)
10	273,9	10	173,7
20	550,2	20	307,8
30	784,6	30	443,3
40	1051,0	40	586,7
50	1327,1	50	722,1
60	1562,0	60	857,7
70	1828,2	70	1000,2
80	2105,1	80	1135,6
90	2339,6	90	1271,7
100	2605,8	100	1406,6

Tabelle 5.5: Hinzufügen der Paritätsbits pro Block und über alle Blöcke alle zehn Blöcke.

Die Tabelle 5.6 wird die Taktanzahl für das Entfernen der Paritätsbits dargestellt. Die Taktanzahl steigt für den Hamming-Code pro Block und über alle Blöcke jeweils linear an. Jedoch gibt es hier wie bei Tabelle 5.5 Schwankungen. Dabei ist für den Hamming-Code pro Block zu beachten, dass alle 16 Blöcke ein weiterer zusätzlicher Block benötigt wird. Für den Hamming-Code über alle Blöcke ist dies nicht der Fall, wodurch die Taktanzahl tendenziell langsamer steigt. Es wird aus den gleichen Gründen wie bei Tabelle 5.5 über eine Blockanzahl von zehn bis 100 Blöcken iteriert.

Paritätsbits entfernen			
Messungen = 30			
pro Block		über alle Blöcke	
Blockanzahl	Takte (Tsd.)	Blockanzahl	Takte (Tsd.)
10	90,4	10	90,7
20	180,5	20	176,2
30	265,6	30	261,7
40	355,6	40	347,1
50	445,7	50	432,6
60	530,8	60	518,1
70	620,9	70	603,6
80	711,0	80	689,0
90	796,1	90	774,5
100	872,5	100	859,9

Tabelle 5.6: Entfernen der Paritätsbits pro Block und über alle Blöcke alle zehn Blöcke.

Dabei dauert das Hinzufügen der Paritätsbits länger, da dafür mehr Schritte als beim Entfernen benötigt werden.

Die Verifizierung des Hamming-Codes pro Block lässt sich in Tabelle 5.7 betrachten. Es werden verschiedene Fehlerarten untersucht und dazu jeweils unterschiedliche Parameter variiert. Die ersten vier Messreihen zeigen die Verifizierung für eine steigende Blockanzahl. Die Bitfehler befinden sich dabei immer im ersten Block. Dabei fällt auf, dass alle Fehlerarten immer ähnlich lange brauchen. Es spielt somit die Art des Fehlers keine signifikante Rolle. All diese Fehlerarten steigen für eine steigende Blockanzahl linear an. Der unauffindbare Fehler ist ein 3-Bit-Fehler und verhält sich wie der 1-Bit-Fehler. Deshalb wird dieser in den weiteren Messreihen der Tabelle 5.7 nicht weiter beachtet. Bei den nächsten zwei Messreihen variiert der Index des fehlerhaften Blocks. Es spielt dabei keine Rolle, in welchem Block sich der Bitfehler befindet. In den letzten zwei Messreihen dieser Tabelle wird die Anzahl der Blöcke mit Bitfehlern variiert. Die Messung ist so aufgebaut, dass zu Beginn nur der erste Block betroffen ist. In der nächsten Iteration die ersten zwei Blöcke und so weiter. Wie man sehen kann, spielt die Anzahl der Blöcke mit Bitfehlern keine Rolle.

In der Tabelle 5.8 sieht man die Verifizierung des Hamming-Codes über alle Blöcke. Wie in Tabelle 5.7 werden verschiedene Fehlerarten mit unterschiedlichen Parametern betrachtet. Zunächst verändert sich in den ersten vier Messreihen die Blockanzahl. Alle Fehlerarten steigen dabei linear an und dauern immer ähnlich lange. Beim unauffindbaren Fehler wird wieder ein 3-Bit-Fehler genutzt, wodurch sich dieser wie ein 1-Bit-Fehler verhält. In den anderen zwei Messreihen wird der fehlerhafte Block variiert. Das hat jedoch keine Auswirkungen auf die Taktanzahl.

Wie man im Vergleich zwischen Tabelle 5.9 und Tabelle 5.4 bereits sehen kann, braucht der Hamming-Code weniger Takte als der MAGIC-Modus. Tendenziell steigt die Taktanzahl in Tabelle 5.9 mit einem schwankenden Faktor linear an.

Hamming-Code pro Block mit Tokengenerierung verifizieren												
no error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,9	1,2	1,5	1,8	2,1	2,4	2,7	3,0	3,3
one bit error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,9	1,2	1,5	1,9	2,2	2,5	2,8	3,1	3,4
(1. Block)												
two bit error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,8	1,2	1,5	1,8	2,1	2,4	2,7	3,0	3,4
(1. Block)												
undetactable error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,8	1,2	1,5	1,9	2,2	2,5	2,8	3,1	3,4
(1. Block)												
one bit error	Blockanzahl = 10 Messungen = 30	Index fehlerhafter Block	0	1	2	3	4	5	6	7	8	9
		Takte (Mio.)	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4
two bit error	Blockanzahl = 10 Messungen = 30	Index fehlerhafter Block	0	1	2	3	4	5	6	7	8	9
		Takte (Mio.)	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4
one bit error	Blockanzahl = 10 Messungen = 30	Anzahl Blöcke mit Bitfehler	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	3,5	3,5	3,5	3,5	3,5	3,5	3,5	3,5	3,5	3,5
two bit error	Blockanzahl = 10 Messungen = 30	Anzahl Blöcke mit Bitfehler	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4

Tabelle 5.7: Verifizierung des Hamming-Codes pro Block mit verschiedenen Parametern und Fehlerarten.

Hamming-Code über alle Blöcke mit Tokengenerierung verifizieren												
no error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,9	1,2	1,5	1,8	2,1	2,4	2,7	3,0	3,3
one bit error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,9	1,2	1,5	1,8	2,1	2,4	2,8	3,1	3,4
(1. Block)												
two bit error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,9	1,2	1,5	1,8	2,1	2,4	2,7	3,0	3,4
(1. Block)												
undetactable error	Messungen = 30	Blockanzahl	1	2	3	4	5	6	7	8	9	10
		Takte (Mio.)	0,6	0,9	1,2	1,5	1,8	2,1	2,4	2,8	3,1	3,4
(1. Block)												
one bit error	Blockanzahl = 10 Messungen = 30	Index fehlerhafter Block	0	1	2	3	4	5	6	7	8	9
		Takte (Mio.)	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4
two bit error	Blockanzahl = 10 Messungen = 30	Index fehlerhafter Block	0	1	2	3	4	5	6	7	8	9
		Takte (Mio.)	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4	3,4

Tabelle 5.8: Verifizierung des Hamming-Codes über alle Blöcke mit verschiedenen Parametern und Fehlerarten.

Hamming-Code kompletter Durchlauf mit Tokengenerierung			
Messungen = 30			
pro Block		über alle Blöcke	
Blockanzahl	Takte (Mio.)	Blockanzahl	Takte (Mio.)
1	1,3	1	1,3
2	2,0	2	2,0
3	2,6	3	2,4
4	3,3	4	3,3
5	3,9	5	3,9
6	4,7	6	4,6
7	5,4	7	5,3
8	5,9	8	5,9
9	6,8	9	6,6
10	7,4	10	7,2

Tabelle 5.9: Komplette Durchläufe des Hamming-Codes mit einem 1-Bit-Fehler.

Bei der Performance des Hamming-Codes ist noch anzumerken, dass ab einer Blockanzahl von ungefähr 150 Blöcken der Hamming-Code auf dem Mikrocontroller nicht mehr richtig funktioniert hat. Da dies jedoch auf einem normalen Computer nicht aufgetreten ist, liegt es vermutlich an den beschränkten Ressourcen des Mikrocontrollers. Wahrscheinlich reicht der Speicher nicht aus, da weitere `bool`-Arrays genutzt werden, um die Bit-Repräsentationen der verschlüsselten Blöcke zu speichern. Ein Array benötigt somit mindestens $n \cdot 128$ Bytes Platz, wobei n für die Anzahl der Blöcke steht. Dazu kommt, dass mehrere solche Arrays vor allem beim Hinzufügen der Paritätsbits genutzt werden.

5.5 Performance der Ver- und Entschlüsselung

Hier wird die Performance der Erzeugung von verschlüsselten Blöcken (Verschlüsselung) und deren Rückführung (Entschlüsselung) durch Salsa20 ermittelt. Es wird ein String in verschlüsselte Blöcke umgewandelt. Bei der Entschlüsselung geschieht dies andersherum. Diese verschlüsselten Blöcke werden vom MAGIC-Modus und vom Hamming-Code genutzt. In der Tabelle 5.10 sieht man die Ergebnisse. Wie man im Vergleich zu den anderen Tabellen feststellen kann, geht beides sehr schnell. Im Prinzip steigt die Taktanzahl bei einer steigenden Blockanzahl linear an. Man kann in der Tabelle 5.10 aber feststellen, dass es alle vier Blöcke einen größeren Sprung gibt. Dies liegt daran, dass Salsa20 mit 512-Bit großen Blöcken arbeitet. Also ein 32-Bit-Array mit 16 Feldern. Daneben entspricht ein Block in der genannten Tabelle 128-Bit. Somit muss alle vier Blöcke ein neuer 512-Bit großer Block erzeugt werden, sodass dieser Sprung alle vier

Blöcke entsteht.

Ver- und Entschlüsselung durch Salsa20			
Messungen = 50			
Verschlüsselung		Entschlüsselung	
Blockanzahl	Takte (Tsd.)	Blockanzahl	Takte (Tsd.)
1	5,7	1	4,7
2	7,1	2	5,5
3	8,4	3	6,2
4	9,7	4	7,0
5	14,9	5	11,2
6	16,3	6	12,0
7	17,6	7	12,8
8	19,0	8	13,5
9	24,2	9	17,8
10	25,5	10	18,6

Tabelle 5.10: Die Anzahl der Takte für die Ver- und Entschlüsselung mit Salsa20.

5.6 Speicherkosten und Mehraufwand der beiden Verfahren

MAGIC-Modus Es werden 128-Bit große Blöcke für MAGIC verwendet. Zu MAGIC gehören vor allem die verschlüsselten Blöcke und der Token. Dabei spielt die Anzahl der verschlüsselten Blöcke für die Größe des Tokens keine Rolle. Dieser ist immer 128-Bit groß. Somit braucht man für die authentifizierte Verschlüsselung und Fehlerkorrektur immer nur 128 zusätzliche Bits, da der Token auch für die Fehlerkorrektur eingesetzt wird. Je mehr verschlüsselte Blöcke es gibt, desto besser wird das Verhältnis zwischen diesen Blöcken und dem Token. Des Weiteren muss der Hashwert, die autorisierten Daten, ein Schlüssel und eine Nonce gespeichert werden, um die `verify`-Funktion nutzen zu können. Um den Klartext auch wiederherzustellen, muss ebenfalls ein zusätzlicher Schlüssel und Nonce gespeichert werden. Dadurch entsteht neben den verschlüsselten Blöcken ein konstanter Mehraufwand von insgesamt 1104-Bit.

Hamming-Code In dieser Arbeit werden für den Hamming-Code 128-Bit-Blöcke verwendet. Man könnte auch andere Blockgrößen dafür nutzen. Um die Fehlerkorrektur pro Block sicherzustellen werden pro Block acht zusätzliche Bits benötigt. Es muss auch ein weiterer Block hinzugefügt werden, sodass alle Bits Platz haben. Außerdem wird für alle 16 Blöcke noch ein weiterer Block benötigt. Die Gründe kann man im Abschnitt 4.6 nachlesen. Somit werden zum Beispiel für eigentlich 16 Blöcke nun 18 Blöcke benötigt, um die Fehlerkorrektur durch den Hamming-Code pro Block zu gewährleisten.

Das Verhältnis zwischen den Paritätsbits und den eigentlichen Daten wird somit bei einer Erhöhung der verschlüsselten Blöcke nicht wirklich besser. Man braucht alle 16 verschlüsselten Blöcke somit 128 zusätzliche Bits. Wie beim MAGIC-Modus braucht man zunächst 1104 zusätzliche Bits für die Tokengenerierung, Fehlerkorrektur und Entschlüsselung. Daneben kommen noch mindestens 128 Bits für einen zusätzlichen Block hinzu, sodass die Paritätsbits auch Platz haben. Dadurch entsteht ein Mehraufwand von mindestens 1232 Bits, wobei alle 16 Blöcke nochmal 128 Bits dazukommen.

Wenn man den Hamming-Code aber über alle Blöcke implementiert, werden immer $\log_2(n) + 1$ zusätzliche Paritätsbits benötigt. Dies entspricht einem zusätzlichen Block. Wie man anhand von Abschnitt 4.6 sieht, bleibt dies auch für eine höhere Blockanzahl so. Je mehr verschlüsselte Blöcke genutzt werden, desto besser wird das Verhältnis zwischen den verschlüsselten Blöcken und den zusätzlichen Daten für die Fehlerkorrektur. Man braucht für die zusätzlichen Paritätsbits immer nur 128 zusätzliche Bits. Da ebenfalls der Token aus bekannten Gründen verwendet wird, entsteht ein konstanter Mehraufwand von 1232 Bits.

Kapitel 6

Schlussfolgerung

In diesem Kapitel werden die Schlüsse aus dem vorherigen Kapitel 5 gezogen. Dazu werden die Ergebnisse des MAGIC-Modus und des Hamming-Codes miteinander verglichen.

6.1 Die Performance im Vergleich

Wie man anhand der Tabellen aus dem vorherigen Kapitel 5 sehen kann, schneidet der Hamming-Code in Sachen Geschwindigkeit deutlich besser als der MAGIC-Modus ab. Der größte Anteil beim Hamming-Code hat die Tokengenerierung. Insbesondere reicht für die Tokengenerierung beim Hamming-Code ein beliebiger Hashwert aus. Deshalb ist dieser deutlich schneller als der MAGIC-Modus bei der Fehlerkorrektur. Anstatt einen bestimmten Hashwert zu finden, muss man beim Hamming-Code Paritätsbits hinzufügen und einen beliebigen Hashwert nutzen. Das Hinzufügen und Entfernen der Paritätsbits fällt so gut wie gar nicht beim Hamming-Code ins Gewicht. Bei beiden Verfahren steigt die Taktanzahl immer linear an oder bleibt bei manchen Parametern gleich.

Beim MAGIC-Modus fällt vor allem auf, dass das Finden eines Hashwertes den größten Anteil bei der Performance hat. Selbst für eine kleine Blockanzahl und einem Schwellenwert von eins dauert diese Operation sehr lange. Schon ab einem Schwellenwert von zwei bei nur einem Block hat das Finden eines passenden Hashwertes zu lange gedauert, um es in der Praxis einsetzen zu können. Vor allem auf dem Mikrocontroller. Die lange Ausführungszeit entsteht dadurch, dass über alle möglichen Fehlervektoren unterhalb eines Schwellenwertes iteriert werden muss. Man müsste das Finden eines Hashwertes optimieren oder diese Werte im Voraus auf einem leistungsstarken Computer berechnen. Jedoch könnte darin auch ein Sicherheitsrisiko bestehen, da die möglichen Hashwerte schon bekannt wären. Auch die Verifizierung der verschlüsselten Blöcke hat bei MAGIC deutlich länger gedauert. Dies liegt vor allem daran, dass beim Hamming-Code nur XOR-Operationen für die Fehlerkorrektur zum Einsatz kommen. Dagegen wird bei MAGIC in großen Teilen die Arithmetik in endlichen Körpern

verwendet, um Fehler zu korrigieren.

Wenn die Bestimmung eines passenden Hashwertes effizienter geschieht, spricht nichts dagegen dieses Verfahren in der Praxis auf einem Mikrocontroller einzusetzen. Die restlichen Operationen dauern zwar alle länger als beim Hamming-Code, jedoch alles noch in einem normalen Zeitumfang. Insbesondere ist beim MAGIC-Modus zu beachten, dass dieser viel mehr Fehler als der Hamming-Code korrigieren kann und gleichzeitig auch eine authentifizierte Verschlüsselung bietet. Des Weiteren bleibt die Nachricht unverändert, da keine weiteren Paritätsbits gebraucht werden.

6.2 Speicherkosten und Mehraufwand im Vergleich

Wie man anhand von Abschnitt 5.6 sehen kann, braucht der Hamming-Code mindestens 128 Bit mehr Speicherplatz als der MAGIC-Modus. Dies gilt aber nur für den Hamming-Code über alle Blöcke, da bei der anderen Variante alle 16 Blöcke nochmal 128 zusätzliche Bits benötigt werden. Insbesondere hat der MAGIC-Modus den Vorteil, dass die Speicherkosten, die durch die Tokengenerierung entstehen immer konstant sind. Dies ist auch nur beim Hamming-Code über alle Blöcke der Fall. Des Weiteren ist das Verhältnis zwischen den zusätzlichen Daten und den verschlüsselten Blöcken beim MAGIC-Modus und dem Hamming-Code über alle Blöcke immer gleich. Beim Hamming-Code pro Block wird dieses Verhältnis im Vergleich mit dem MAGIC-Modus nicht besser.

Außerdem kann man aussagen, dass die Anzahl der verschlüsselten Blöcke keine nennenswerten Auswirkungen auf die Fehlerkorrektur und Sicherheit hat. Die Fehlererkennung geschieht nur schneller, wenn weniger Blöcke vorhanden sind, was man anhand von Tabelle 5.3 sehen kann. Jedoch ist die Wahrscheinlichkeit bei einer geringen Blockanzahl höher, einen Fehler erfolgreich zu verbessern, da es weniger Stellen gibt, bei denen ein Bitfehler auftreten kann.

Kapitel 7

Fazit und Ausblick

In diesem Kapitel folgt ein Rückblick auf das Einführungskapitel 1. Anhand dessen wird ein Fazit gezogen. Des Weiteren folgt ein Ausblick, indem mögliche Weiterentwicklungen und Verbesserungen vorgestellt werden.

7.1 Fazit

Im Allgemeinen kann man sagen, dass der Hamming-Code deutlich schneller als der MAGIC-Modus in Bezug auf die Fehlerkorrektur ist. Dabei hat das Finden des Hashwertes den größten Anteil am MAGIC-Modus. Würde dies schneller gehen, wäre MAGIC ein sehr geeignetes Verfahren auf dem Mikrocontroller. Vor allem sind die anderen Bestandteile immer schon im Sekundenbereich fertig. Außerdem lassen sich mit dem MAGIC-Modus in der Theorie viel mehr Bitfehler korrigieren, wenn man einen passenden Hashwert gefunden hat. Beim Hamming-Code ist dies nur möglich, wenn man diesen pro Block anwendet. Jedoch wird dafür mehr Speicherplatz ab einer Blockanzahl von 16 benötigt. Sollte die Blockanzahl darunter bleiben, ist der Hamming-Code pro Block ein sehr geeignetes Verfahren auf dem Mikrocontroller. Jedoch wird immer noch mehr Speicherplatz als beim MAGIC-Modus benötigt und die verschlüsselten Blöcke müssen verändert werden.

Der MAGIC-Modus hat außerdem den Vorteil, dass die Komplexität geringer als beim Hamming-Code ist. Man braucht nur einen Token und passenden Hashwert, um Fehler zu korrigieren und zu finden. Die verschlüsselten Blöcke bleiben dabei unverändert. Dagegen müssen beim Hamming-Code Paritätsbits in die verschlüsselten Blöcke eingefügt werden, wodurch sich diese verändern. Bevor man diese wieder sinnvoll nutzen kann, müssen in einem weiteren Schritt die Paritätsbits entfernt werden. Um die Authentizität beim Hamming-Code sicherzustellen, muss ein Token generiert werden, der nur dafür verwendet wird. Für die Fehlerkorrektur ist dieser nutzlos. Wie man sieht, ist die Komplexität beim MAGIC-Modus geringer, da der Token für die Fehlerkorrektur und Authentifizierung verwendet werden kann. Insbesondere lässt sich der MAGIC-Modus

auch um das Memory-Tagging erweitern, sodass gleichzeitig authentifizierte Verschlüsselung, Fehlerkorrektur und Memory-Tagging in einem Verfahren unterstützt werden. Lamster u. a. haben dies in einem Verfahren namens Voodoo vorgestellt [Lam+24]. Dadurch nimmt wie schon erwähnt die Komplexität ab, da man ansonsten getrennte Verfahren nutzen müsste. Jedoch werden mehr Ressourcen als beim Hamming-Code verwendet, da die Ausführungszeit vom MAGIC-Modus länger ist. In Bezug auf den Speicher werden jedoch weniger Ressourcen benötigt.

Schlussendlich kann gesagt werden, dass der MAGIC-Modus aktuell noch zu lange auf einem Mikrocontroller braucht. Das liegt vor allem an der Überprüfung und dem Finden eines Hashwertes für einen Schwellenwert ab zwei. Sollte dies verbessert oder sollte dies ausreichen, spricht nichts dagegen, MAGIC auf einem Mikrocontroller einzusetzen. Man muss zwischen der Komplexität mit dem geringen Speicheraufwand und der Geschwindigkeit abwägen. Ist die Geschwindigkeit wichtig, sollte man den Hamming-Code einsetzen. Wenn jedoch die Komplexität und der Speicheraufwand sehr wichtig sind, könnte man den MAGIC-Modus bei einer geringen Blockanzahl einsetzen. Mit den hier vorgestellten Mitteln lassen sich sogar 2-Bit-Fehler in einem noch überschaubaren Zeitaufwand korrigieren, was beim Hamming-Code so nicht direkt möglich ist. Man muss also selbst entscheiden, ob es einem Wert ist, 128 zusätzliche Bits zu sparen und dafür einen sehr hohen Zeitaufwand in Kauf zu nehmen.

7.2 Ausblick und mögliche Weiterentwicklungen

Der größte Gewinn beim MAGIC-Modus wäre es, das Finden eines Hashwertes zu optimieren. Der Grund für die lange Ausführung besteht darin, dass über alle Fehlervektoren mit einem Hamming-Gewicht gleich und kleiner des Schwellenwertes iteriert werden muss. Man müsste für die Überprüfung des Hashwertes vermutlich meiden, dass man alle Fehlervektoren unter einem bestimmten Schwellenwert dafür benötigt. Es ist auch möglich, Hashwerte im Voraus auf einem leistungsstarken Computer zu berechnen und diese anschließend nur auf dem Mikrocontroller zu speichern. Jedoch erhöht sich dadurch der Speicheraufwand. Man müsste diesbezüglich ebenfalls abwägen, ob dies aus Sicherheitsgründen akzeptabel wäre.

Des Weiteren könnte man den erweiterten euklidischen Algorithmus anwenden, um die multiplikativ inversen Elemente schneller in endlichen Körpern zu finden [Bab]. Aktuell wird dafür der kleine Satz von Fermat in Kombination mit der Multiplikation von Zweierpotenzen angewendet. Dadurch könnte der MAGIC-Modus an vielen Stellen schneller werden, da man oft das multiplikative Inverse, bzw. große Potenzen berechnen muss. Des Weiteren könnte man auch Additionsketten anwenden, um die Potenzen schneller zu berechnen [Cli]. Vor allem würde die Tokengenerierung von einer schnelleren Potenzierung profitieren.

Außerdem könnte für den Hamming-Code eine effizientere Implementierung genutzt werden, die auf zusätzliche Bit-Repräsentationen verzichtet. So könnte man eventuell auch eine Anzahl von Blöcken größer als 150 nutzen. Ob man überhaupt 150 Blöcke auf einen Schlag nutzen möchte, ist eine andere Frage.

An dieser Stelle möchte ich mich bei allen bedanken, die mich in den letzten drei Monaten unterstützt haben.

Besonders möchte ich mich bei Herr Professor Dr. STÖTTINGER und Herr Wälde M. Sc. bedanken, die diese Arbeit betreut haben und immer ein offenes Ohr für Fragen hatten.

Außerdem möchte ich mich bei meiner Familie und Freunden bedanken, die mich während des kompletten Studiums unterstützt haben. Ohne sie hätte ich das nicht geschafft.

Informationen zum Quellcode und zu Quellenangaben

Der Quellcode ist auf GitHub unter folgendem Link zu finden: https://github.com/Jonas0503/Bachelorarbeit_MAGIC

Quellenangaben am Ende eines Absatzes gelten für den kompletten Absatz. Sollte eine Quellenangabe innerhalb eines Absatzes vorkommen, gilt diese nur für den jeweiligen Satz. Ansonsten sollten andere Formen der Quellenangabe durch den Text ersichtlich sein.

Abkürzungsverzeichnis

AES	Advanced Encryption Standard
DEMCR	Debug Exception and Monitor Control Register
DWT	Data Watchpoint and Trace
GCM	Galois Counter Mode
GMAC	Galois Message Authentication Code
JTAG	Joint Test Action Group
IoT	Internet of Things
KAT	Known Answer Test
LSB	niedrigstwertige Bit
MAC	Message Authentication Code
MAGIC	Message Authentication, Galois Integrity and Correction
MSB	höchstwertige Bit
RAM	Random Access Memory
SRAM	Static Random Access Memory
UML	Unified Modeling Language

Abbildungsverzeichnis

2.1	Die Tokengenerierung des MAGIC Modus [USE24].	6
2.2	Der initiale Zustand eines Blocks [SKS22]. Die Felder bilden zeilenweise den Array von Index null bis 15.	8
4.1	Der grundlegende Aufbau der MAGIC und Hamming-Code Implementie- rung.	16

Tabellenverzeichnis

2.1	Beispielhafte Darstellung des Hamming-Codes.	10
5.1	Anzahl der Takte in Millionen zum Finden eines Hashwertes.	40
5.2	Tockengenerierung für unterschiedlich große Blockzahlen.	41
5.3	Die Takte, um MAGIC mit unterschiedlichen Fehlerarten und Variationen zu verifizieren.	42
5.4	Ein kompletter Durchlauf des MAGIC-Modus mit einem 1-Bit-Fehler. . . .	43
5.5	Hinzufügen der Paritätsbits pro Block und über alle Blöcke alle zehn Blöcke.	44
5.6	Entfernen der Paritätsbits pro Block und über alle Blöcke alle zehn Blöcke.	44
5.7	Verifizierung des Hamming-Codes pro Block mit verschiedenen Parame- tern und Fehlerarten.	46
5.8	Verifizierung des Hamming-Codes über alle Blöcke mit verschiedenen Parametern und Fehlerarten.	47
5.9	Komplette Durchläufe des Hamming-Codes mit einem 1-Bit-Fehler. . . .	48
5.10	Die Anzahl der Takte für die Ver- und Entschlüsselung mit Salsa20. . . .	49

Quellcodeverzeichnis

4.1	Die Darstellung einer 128-Bit-Zahl.	17
4.2	Die Umsetzung einer Shift-Left-Operation für bignum.	18
4.3	Die Multiplikation in $GF(2^{128})$	20
4.4	Eine große Potenz berechnen, um das inverse Element zu finden.	20
4.5	Die Shift-Left-Rotate Operation für Salsa20.	21
4.6	Das Prinzip der Ver- und Entschlüsselung von Salsa20.	22
4.7	Zufällige Werte mit Salsa20 erzeugen.	23
4.8	Alle Kombinationen unter einem Schwellenwert von Fehlervektoren fin- den und damit den Hashwert überprüfen.	25
4.9	Der Rückgabetypp für die verify-Funktion im MAGIC-Modus.	26
4.10	Berechnung der einzelnen Syndromwerte.	26
4.11	Die Funktion, um einen fehlerhaften Block möglicherweise zu lokalisieren.	27
4.12	Die Schleife, um Platz für die Paritätsbits über alle Blöcke zu schaffen.	29
4.13	Das Setzen der Paritätsbits über alle Blöcke.	30
4.14	Der Rückgabetypp zum Verifizieren des Hamming-Codes.	31
4.15	Angepasste Zeilen im Makefile des Mikrocontrollers.	33
4.16	Ein beispielhafter Test.	34
4.17	Die Generierung von Testwerten zum Testen der Arithmetik in endlichen Erweiterungskörpern.	34
5.1	Die Funktion, um die Register zu konfigurieren.	37
5.2	Eine repäsentative Ermittlung der Takte für eine Operation.	38

Literatur

- [3Bl20] 3Blue1Brown. *Hamming codes part 2: The one-line implementation*. YouTube. Abgerufen: 11.04.2025. 2020. URL: https://youtu.be/b3NxrZ0u_CE?si=edGJpii_4ooAhP-0.
- [Asp14] James Aspnes. *Generating random values in C*. Abgerufen: 07.04.2025. 2014. URL: [https://www.cs.yale.edu/homes/aspnes/pinewiki/C\(2f\)Randomization.html](https://www.cs.yale.edu/homes/aspnes/pinewiki/C(2f)Randomization.html).
- [Bab] University of Babylon. *Modular multiplicative inverse*. Abgerufen: 22.04.2025. URL: https://www.uobabylon.edu.iq/eprints/publication_11_21700_649.pdf.
- [Ber05] Daniel J. Bernstein. "Salsa20 specification". In: *eSTREAM Project algorithm description* (2005). URL: <https://cr.yp.to/snuffle/spec.pdf>.
- [Ber08] Daniel J. Bernstein. "The Salsa20 Family of Stream Ciphers". In: *New Stream Cipher Designs: The eSTREAM Finalists*. Hrsg. von Matthew Robshaw und Olivier Billet. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, S. 84–97. ISBN: 978-3-540-68351-3. DOI: 10.1007/978-3-540-68351-3_8. URL: https://doi.org/10.1007/978-3-540-68351-3_8.
- [Boc12] Das Labor Hackspace Bochum. *salsa20-256.64-verified.test-vectors*. GitHub Verzeichnis. Abgerufen: 26.03.2025. 2012. URL: <https://github.com/das-labor/legacy/blob/master/microcontroller-2/arm-crypto-lib/testvectors/salsa20-256.64-verified.test-vectors>.
- [Bue21] Kevin Buettner. *Printf-style debugging using GDB*. Abgerufen: 26.03.2025. 2021. URL: <https://developers.redhat.com/articles/2021/10/13/printf-style-debugging-using-gdb-part-2#>.
- [Cev18] Bekir Cevizci. "How and Why Does the Multiplication Method Developed by the Russian Peasants Work?." In: *Journal of Inquiry Based Activities* 8.1 (2018), S. 24–36. URL: <https://eric.ed.gov/?id=EJ1341374>.
- [Cli] Neill Clift. *Addition Chains*. Abgerufen: 28.04.2025. URL: <http://additionchains.com/>.
- [Deva] Arm Developer. *Debug Exception and Monitor Control Register*. Abgerufen: 14.04.2025. URL: <https://developer.arm.com/documentation/ddi0337/e/CEGHJDCF>.

- [Devb] Arm Developer. *Summary and description of the DWT registers*. Abgerufen: 14.04.2025. URL: <https://developer.arm.com/documentation/ddi0337/e/ch11s05s01?lang=en>.
- [Ham50] R. W. Hamming. "Error Detecting and Error Correcting Codes". In: *The Bell System Technical Journal* (1950). URL: <https://zoo.cs.yale.edu/classes/cs323/doc/Hamming.pdf>.
- [HMK18] HMKCODE. *Algorithms for Finding all Possible Combinations of k Elements in an Array with Java*. Abgerufen: 07.04.2025. 2018. URL: <https://hmkcode.com/calculate-find-all-possible-combinations-of-an-array-using-java/>.
- [Inc22] NewAE Technology Inc. *ChipSHOUTER-ballisticgel*. GitHub Verzeichnis. Abgerufen: 26.03.2025. 2022. URL: <https://github.com/newaetech/ChipSHOUTER-ballisticgel>.
- [Kak25] Avi Kak. "PART 4: Finite Fields of the Form $GF(2^n)$ Theoretical Underpinnings of Modern Cryptography". Abgerufen: 24.03.2025. 2025. URL: <https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture7.pdf>.
- [Kou+20] Michael Kounavis u. a. *The MAGIC Mode for Simultaneously Supporting Encryption, Message Authentication and Error Correction*. Cryptology ePrint Archive, Paper 2020/1460. 2020. URL: <https://eprint.iacr.org/2020/1460>.
- [KU07] U. K. Kumar und B. S. Umashankar. "Improved Hamming Code for Error Detection and Correction". In: *2007 2nd International Symposium on Wireless Pervasive Computing*. 2007. DOI: 10.1109/ISWPC.2007.342654.
- [Lam+24] Lukas Lamster u. a. "Voodoo: Memory Tagging, Authenticated Encryption, and Error Correction through MAGIC". In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, S. 7159–7176. ISBN: 978-1-939133-44-1. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/lamster>.
- [Mic] Microcontroller.net. *JTAG*. Abgerufen: 26.03.2025. URL: <https://www.mikrocontroller.net/articles/JTAG>.
- [PP16] Christof Paar und Jan Pelzl. *Kryptografie verständlich. Ein Lehrbuch für Studierende und Anwender*. Springer Vieweg Berlin, Heidelberg, 2016. ISBN: 978-3-662-49297-0. DOI: 10.1007/978-3-662-49297-0. URL: <https://doi.org/10.1007/978-3-662-49297-0>.
- [SKS22] Sylwia Stachowiak, Mirosław Kurkowski und Artur Soboń. "SAT-Based Cryptanalysis of Salsa20 Cipher". In: *Progress in Image Processing, Pattern Recognition and Communication Systems*. Hrsg. von Michal Choraś u. a.

- Cham: Springer International Publishing, 2022, S. 252–266. ISBN: 978-3-030-81523-3. URL: https://doi.org/10.1007/978-3-030-81523-3_25.
- [USE24] USENIX. *USENIX Security '24 - Voodoo: Memory Tagging, Authenticated Encryption, and Error Correction through MAGIC*. YouTube - USENIX. Abgerufen: 04.03.2025. 2024. URL: <https://youtu.be/VD5cNVRnMGc?si=2IEeUwLFB5qwVt9M&t=261>.
- [VK07] J. Ivan Vargas und Olga Kosheleva. *Russian peasant multiplication algorithm, RSA cryptosystem, and a new explanation of half-orders of magnitude*. Techn. Ber. UTEP-CS-07-09. Departmental Technical Reports (CS), 2007. URL: https://scholarworks.utep.edu/cs_techrep/103/.